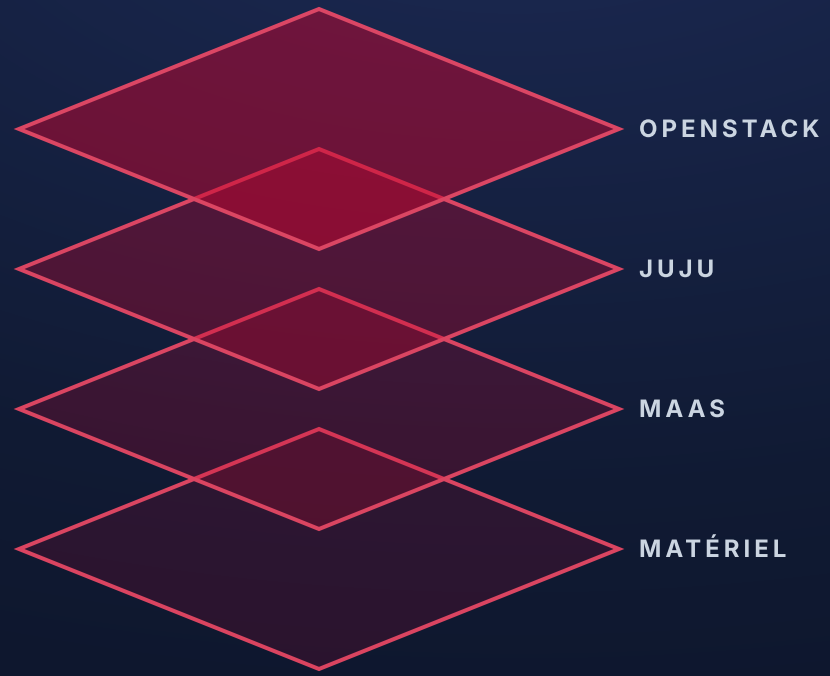
COMPRENDRE OPENSTACK · MODULE 6

La stack Canonical : MAAS, Juju, Canonical OpenStack

Du serveur nu au cloud privé — architecture, fonctionnalités, avantages, limites et modèle de support



MAAS · Juju · Charms · Sunbeam · Canonical Kubernetes · Ubuntu Pro

État des produits vérifié en octobre 2026 · MAAS 3.7 · Juju 3.6 LTS et 4.0 · Canonical OpenStack 2026.1 LTS

INTRODUCTION

Objectif et déroulé

Objectif : comprendre comment les trois briques de Canonical s'empilent pour produire un cloud OpenStack, ce que chacune apporte seule, et ce qu'implique le choix de cette stack en exploitation et en support.

À l'issue de ce module, vous saurez :

- Situer **MAAS**, **Juju** et **Sunbeam** dans les couches d'un déploiement.
- Décrire l'architecture de MAAS et le **cycle de vie** d'une machine, de la découverte à la mise en service.
- Expliquer le modèle de Juju : contrôleur, modèles, applications, unités, **charms** et **relations**.
- Décrire l'architecture de **Canonical OpenStack** : rôles des machines, réseaux, composants.
- Dérouler une installation, avec ou sans MAAS.
- Énoncer les **avantages**, les **limites** et le **modèle de support** de la stack.

1 · Vue d'ensemble
Les couches, les formats de livraison, la philosophie « pilotée par un modèle ».

2 · MAAS
Provisionnement des serveurs : architecture, cycle de vie, réseau, mise en œuvre.

3 · Juju
Orchestration par charms : modèle, relations, Charmhub, commandes.

4 · Canonical OpenStack (Sunbeam)
Architecture, rôles, réseaux, installation, fonctionnalités, exploitation.

5 · Avantages, limites, support
Bilan, cycle de vie des versions, Ubuntu Pro, services, critères d'adoption.

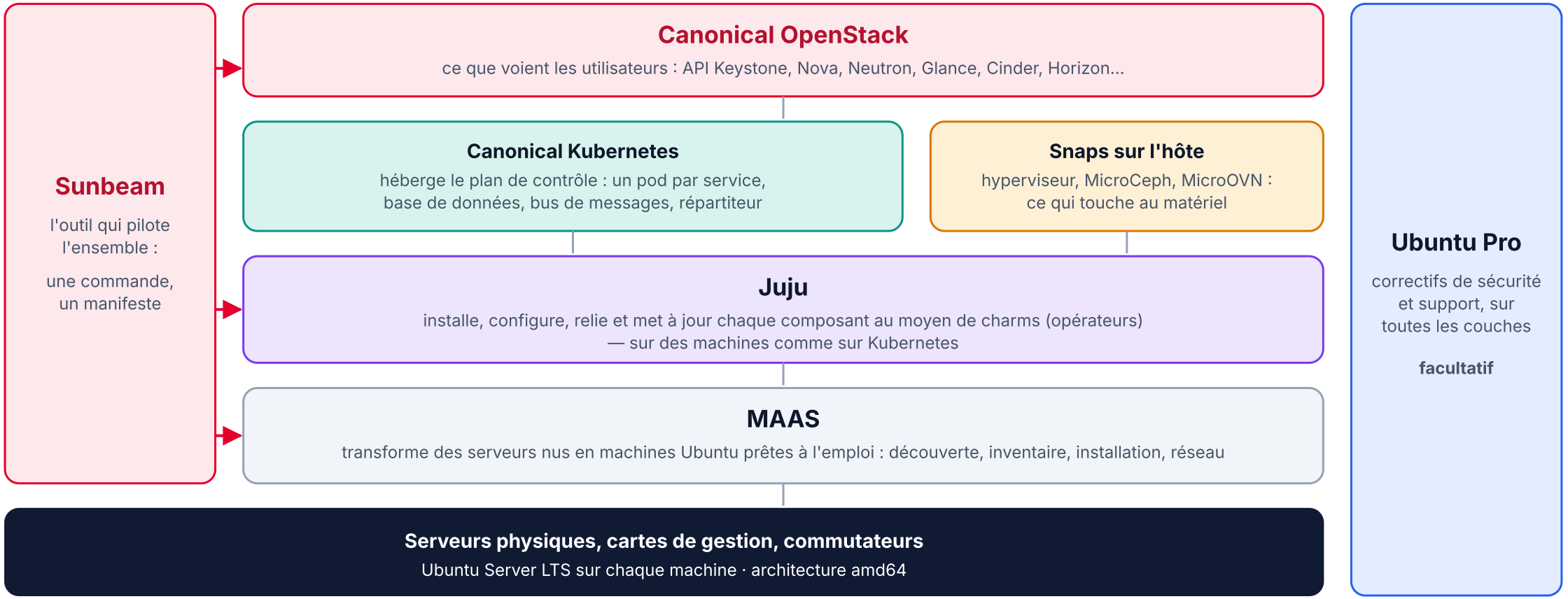
01

Vue d'ensemble de la stack

Trois produits libres, conçus pour s'emboîter : l'un fournit des machines, le deuxième y installe et relie des applications, le troisième assemble un cloud.

- La stack en une image
- Les formats de livraison de Canonical
- L'écosystème autour des trois briques
- Qui fait quoi
- Des opérations pilotées par un modèle

La stack en une image



Lecture : chaque brique est utilisable seule — MAAS sans Juju, Juju sans MAAS ni OpenStack. Sunbeam les enchaîne pour vous ; sur un petit cloud, MAAS est facultatif et les machines sont installées à la main.

1 · VUE D'ENSEMBLE

Qui fait quoi

Brique	En une phrase	Ce qu'elle prend en entrée	Ce qu'elle rend	Équivalents ailleurs
MAAS	**le « cloud » des serveurs physiques**	des serveurs nus, câblés, avec carte de gestion	des machines Ubuntu installées, à la demande, par API	Ironic, Foreman, Metal3
Juju	**l'orchestrateur d'applications**	des machines ou un cluster Kubernetes, et des charms	des applications installées, configurées et reliées entre elles	Ansible, Helm et opérateurs Kubernetes
Charms	**le savoir-faire d'exploitation, emballé**	des événements envoyés par Juju	les actions à mener : installer, configurer, migrer, mettre à jour	rôles Ansible, charts Helm, opérateurs
Canonical Kubernetes	**le socle du plan de contrôle**	les machines de contrôle	un cluster où tournent les services d'OpenStack	OpenShift, toute distribution Kubernetes
Sunbeam	**l'assembleur du cloud**	des machines, quelques réponses ou un manifeste	un cloud OpenStack complet et ses mises à jour	Kolla-Ansible, RHOSO, MOSK
Ubuntu Pro	**l'abonnement**	un jeton attaché à chaque machine	correctifs étendus, conformité, support en option	souscriptions Red Hat, Mirantis

Tout est libre et gratuit à l'usage : MAAS, Juju, les charms, Sunbeam et Ubuntu se téléchargent et s'utilisent en production sans contrat. L'abonnement ajoute des correctifs sur une durée plus longue et un interlocuteur.

« Canonical OpenStack » désigne aujourd'hui la distribution fondée sur **Sunbeam**. L'ancienne génération, fondée sur les charms « machine », s'appelle **Charmed OpenStack** et reste maintenue pour les clouds existants.

Les formats de livraison de Canonical

Paquet deb

Le format historique d'Ubuntu. Dépendances partagées avec le système, mises à jour par `apt`. Les services OpenStack sont d'abord empaquetés ainsi dans l'archive Ubuntu.

Où : à l'intérieur des rocks et des snaps

Snap

Application confinée, livrée avec ses dépendances, mise à jour de façon transactionnelle par **canaux** avec retour arrière. Indépendante de la version du système.

Où : MAAS, Juju, Sunbeam, hyperviseur, MicroCeph, MicroOVN, Kubernetes

Rock

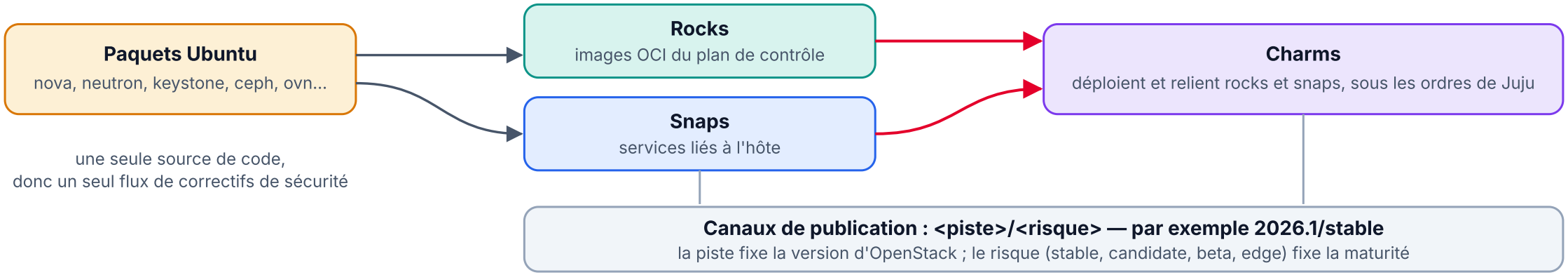
Image de conteneur au format OCI, construite par Canonical à partir de paquets Ubuntu, avec le gestionnaire de processus Pebble.

Où : chaque service du plan de contrôle, dans un pod

Charm

Un opérateur : du code Python qui sait installer, configurer, relier, mettre à l'échelle et mettre à jour une application. Publié sur Charmhub.

Où : un charm par rock et par snap à piloter

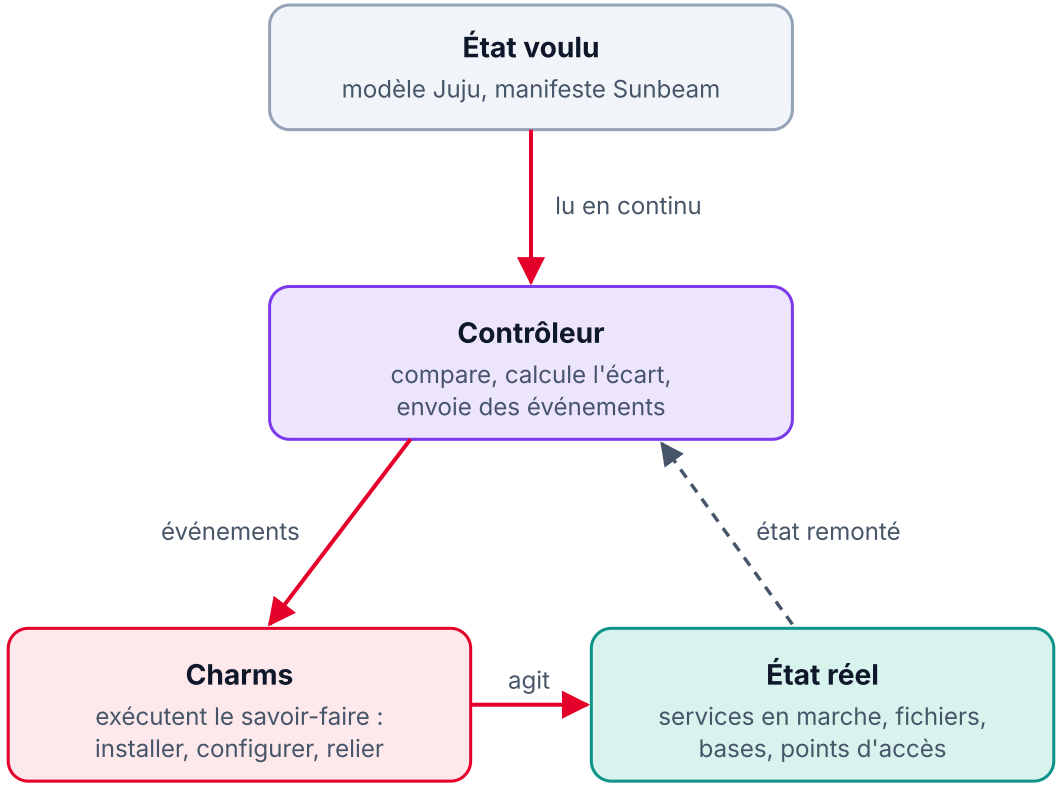


Des opérations pilotées par un modèle

Le point commun des trois briques : on ne décrit pas **les étapes** à exécuter, on décrit **l'état voulu**, et un contrôleur se charge d'y arriver puis de s'y maintenir.

Brique	Ce que l'on déclare	Qui réalise
MAAS	« cette machine doit être déployée en Ubuntu 24.04, avec ce partitionnement et ces interfaces »	les contrôleurs de région et de rack
Juju	« le modèle contient Keystone en trois unités, relié à MySQL »	le contrôleur Juju et les charms
Sunbeam	« ce cloud a ces nœuds, ces rôles, ces fonctionnalités »	Sunbeam, qui pilote Juju

- **Ce que cela change :** pas de fichier de configuration édité à la main sur un serveur ; toute modification passe par le modèle.
- **Ce que cela exige :** faire confiance aux choix intégrés dans les charms, et savoir lire l'état du modèle pour diagnostiquer.



La boucle tourne en permanence, pas seulement à l'installation.

1 · VUE D'ENSEMBLE

L'écosystème autour des trois briques

Composant	Rôle	Place dans Canonical OpenStack	Utilisable seul ?
Ubuntu Server LTS	**système de toutes les machines**	24.04 ou 26.04 pour OpenStack 2026.1	oui
LXD	**conteneurs système et machines virtuelles**	héberge les machines de gouvernance (MAAS, contrôleurs)	oui ; socle de MicroCloud
Canonical Kubernetes	**distribution Kubernetes en un snap**	socle du plan de contrôle	oui
MicroCeph	**Ceph en un snap, mis en grappe simplement**	stockage bloc, image et objet	oui
MicroOVN	**OVN en un snap**	passerelles réseau des nœuds « network »	oui
Observabilité Canonical (COS)	**Prometheus, Loki, Grafana, Alertmanager, en charms**	fonctionnalité optionnelle de supervision	oui
Vault, en charm	**coffre de secrets et autorité de certification**	fonctionnalité optionnelle, TLS des points d'accès	oui
Landscape	**gestion de parc Ubuntu : correctifs, inventaire**	inclus dans Ubuntu Pro	oui

Une même logique partout : un snap pour installer, une mise en grappe en quelques commandes, un charm pour l'intégrer. Apprendre une brique aide à comprendre les autres.

Contrepartie : la stack est très cohérente **à l'intérieur** de l'écosystème Canonical, et peu ouverte à des substitutions — autre système hôte, autre Kubernetes, autre réseau virtuel.

02

MAAS — provisionner les serveurs

« Metal as a Service » : traiter un parc de serveurs physiques comme un cloud, où l'on demande une machine par API et où on la rend quand on n'en a plus besoin.

- Rôle et principe
- Cycle de vie d'une machine
- Déploiement d'un système
- Inventaire, stockage, images
- Disponibilité, versions, bilan
- Architecture : région et rack
- Enrôlement et inventaire, pas à pas
- Le modèle réseau
- Mise en œuvre et commandes

MAAS : rôle et principe

Le problème résolu : installer un système sur cent serveurs, à l'identique, sans clé USB ni console, et pouvoir recommencer à volonté.

- **Découvre** les serveurs qui démarrent sur le réseau et dresse leur **inventaire matériel** : processeurs, mémoire, disques, cartes réseau, micrologiciels.
- **Pilote l'alimentation** par la carte de gestion de chaque serveur (IPMI, Redfish).
- **Installe** un système d'exploitation en quelques minutes : Ubuntu, mais aussi CentOS, RHEL, Oracle Linux, Windows, VMware ESXi.
- **Configure** le stockage (partitions, LVM, RAID logiciel) et le réseau (agrégats, VLAN, ponts) **avant** l'installation.
- **Fournit les services réseau** nécessaires : DHCP, DNS, NTP, amorçage, mandataire de paquets.
- **Expose tout par API** : interface web, ligne de commande, Juju, Terraform.

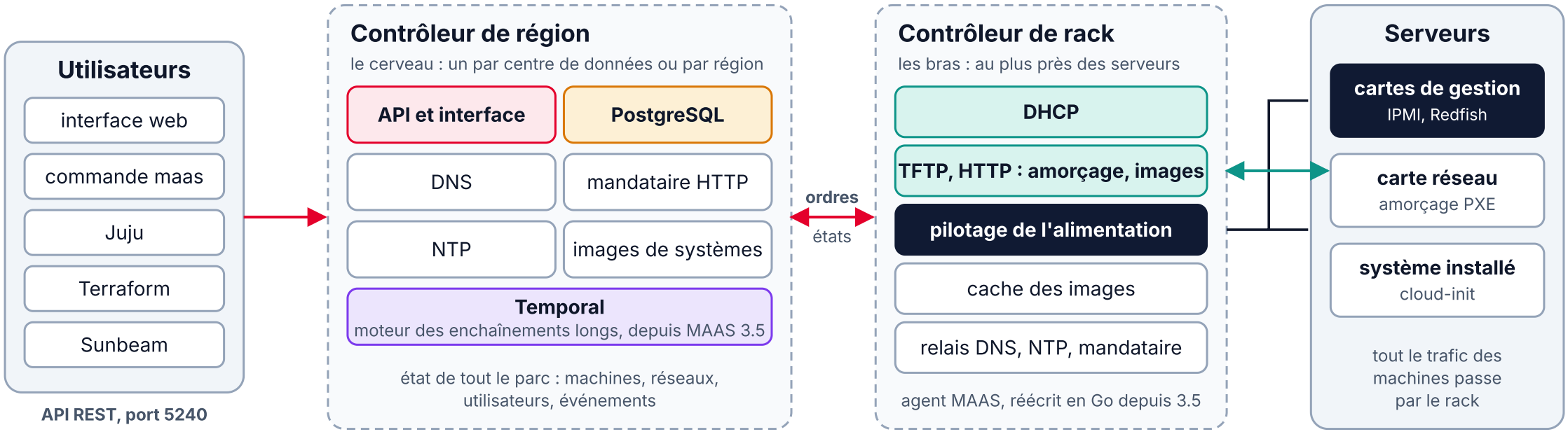
> **Ce que MAAS n'est pas :** un outil de configuration d'applications. Il s'arrête au système installé et joignable en SSH.

Dans un cloud public	Dans MAAS
créer une instance	allouer puis déployer une machine
type d'instance	contraintes : mémoire, cœurs, étiquettes
image	image de système synchronisée
zone de disponibilité	zone
projet, quota	groupe de ressources
données utilisateur	cloud-init
supprimer l'instance	libérer la machine, effacer les disques

> **Pourquoi c'est la base de la stack :** Juju sait parler à MAAS comme à un cloud. Demander « trois machines étiquetées stockage » devient une ligne de commande, et le remplacement d'un nœud en panne une opération banale.

> **Prérequis à vérifier avant tout :** un réseau où MAAS est le **seul** serveur DHCP, l'amorçage réseau activé dans chaque serveur, et un accès aux cartes de gestion.

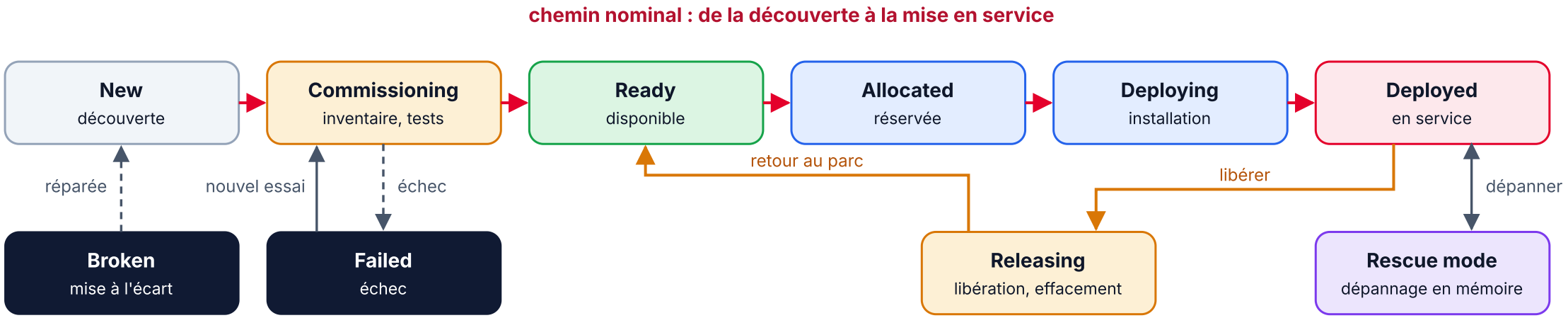
Architecture : contrôleur de région et contrôleurs de rack



Répartition selon la taille

Maquette ou petit site : région et rack sur la même machine (mode « region+rack »), base de test intégrée.
Production : plusieurs régions derrière une base PostgreSQL redondante, et deux contrôleurs de rack par réseau d'amorçage.

Cycle de vie d'une machine

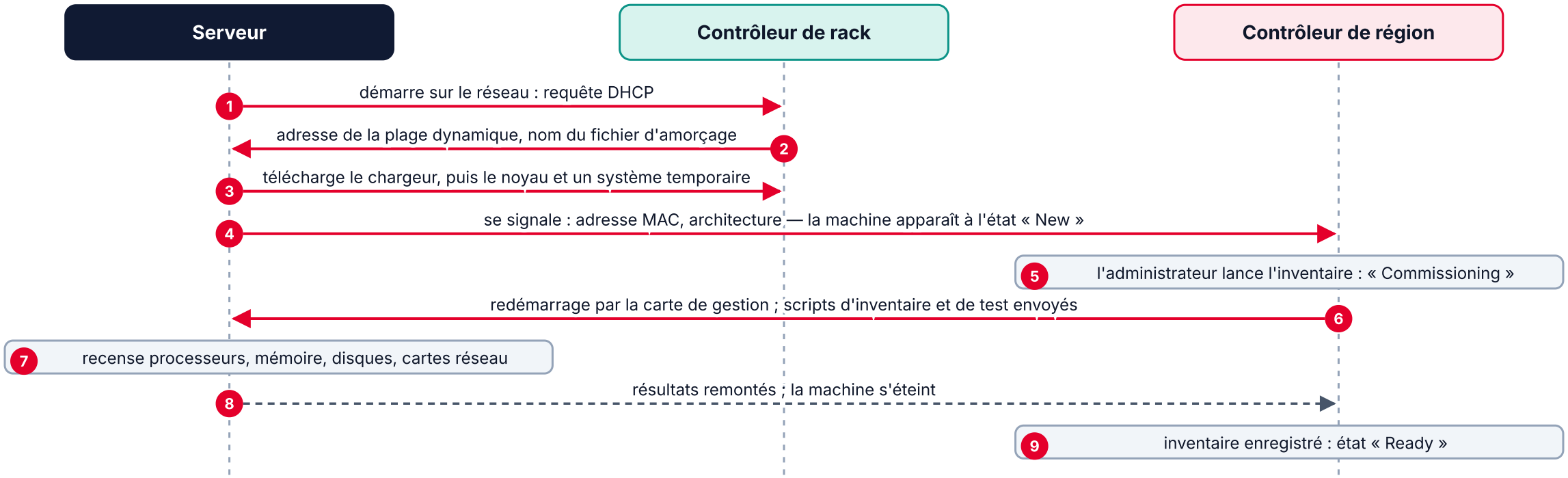


Une machine en panne est marquée « Broken » par l'administrateur ; réparée, elle repasse par l'inventaire.

État	Signification
New	découverte, visible des administrateurs, pas encore de confiance
Commissioning	inventaire matériel et tests sur un système temporaire en mémoire
Ready	inventoriée, inactive, disponible
Allocated	réservée par un utilisateur ; toujours éteinte ou inactive

État	Signification
Deploying, Deployed	installation en cours, puis système en service
Releasing	rendue au parc ; effacement des disques si demandé
Failed	échec d'inventaire, de test ou d'installation : à reprendre
Broken, Rescue mode	mise à l'écart pour panne ; démarrage de dépannage en mémoire

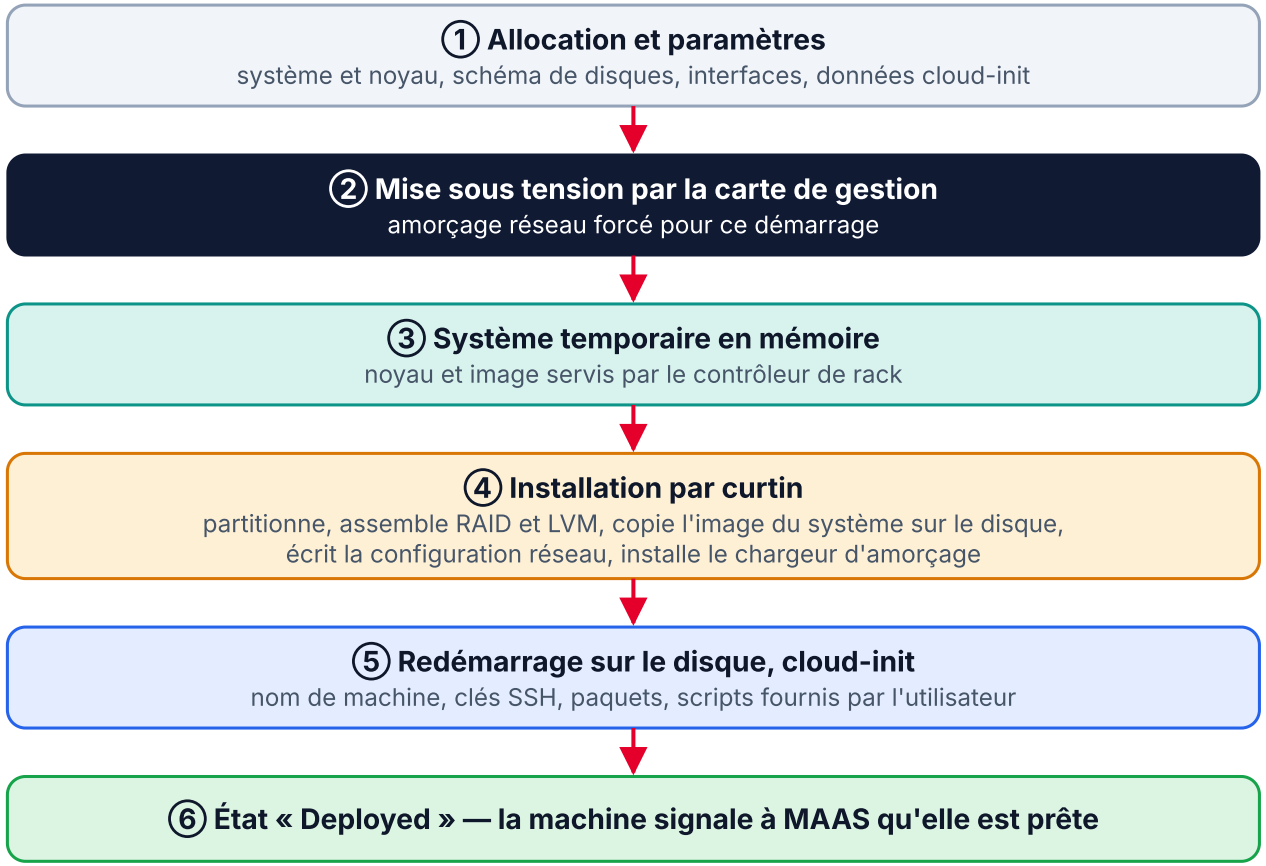
Enrôlement et inventaire, pas à pas



Deux façons d'enrôler : laisser le serveur démarrer sur le réseau, MAAS le découvre seul ; ou le déclarer à la main avec l'adresse de sa carte de gestion, ce qui lance directement l'inventaire.

Premier diagnostic quand rien ne se passe : le serveur a-t-il reçu une adresse ? Un autre serveur DHCP répond-il sur ce VLAN ? L'amorçage réseau est-il en tête dans le micrologiciel ?

Déploiement : de « Ready » à un système en service



Ce qu'il faut retenir

- **Une copie d'image, pas une installation paquet par paquet :** c'est ce qui rend le déploiement rapide et identique d'une machine à l'autre.
- **Le réseau et les disques se décident dans MAAS**, avant le déploiement ; le système arrive déjà configuré.
- **cloud-init** est le point d'entrée de la personnalisation : c'est par lui que Juju installe son agent.
- **Variantes :** déploiement en mémoire seule, sans toucher aux disques (depuis 3.5) ; enregistrement d'une machine déjà installée, sans la réinstaller.

Script	Moment
d'inventaire	pendant « Commissioning » : micrologiciel, RAID matériel
de test	disques, mémoire, réseau
de déploiement	avant ou après l'installation
de libération	au retour dans le parc (depuis 3.5)

Le modèle réseau de MAAS

Fabric « prod » — un ensemble de commutateurs reliés

VLAN non étiqueté

`10.10.0.0/24`

amorçage, gestion

oam

VLAN 100

`10.10.100.0/24`

API internes, base

interne

VLAN 200

`10.10.200.0/24`

accès à Ceph

stockage

Fabric « externe »

VLAN non étiqueté

`192.0.2.0/24`

sortie vers l'extérieur

sous-réseau non géré par MAAS

Vocabulaire

Fabric : domaine de niveau 2

VLAN : segment, avec ou sans DHCP

Sous-réseau : plage IP d'un VLAN

Espace : étiquette d'usage regroupant des sous-réseaux, lue par Juju

Plage réservée : adresses que MAAS ne distribuera jamais

Découpage d'un sous-réseau géré

.1 – .9	.10 – .49	.50 – .99	.100 – .254
passerelle, équipements	plage réservée (adresses virtuelles)	plage dynamique (DHCP)	adresses fixes attribuées aux machines déployées

La plage dynamique sert à l'enrôlement et à l'inventaire. Une machine déployée reçoit en général une adresse **fixe**, choisie par MAAS hors de cette plage et inscrite dans le DNS.

Les espaces sont la clé de l'intégration : Juju et Sunbeam ne manipulent pas des VLAN mais des espaces. « Place ce service sur l'espace interne » fonctionne quel que soit l'adressage du site.

Organiser le parc : étiquettes, zones, disques, images

Notion	À quoi elle sert	Exemple	Usage par Juju et Sunbeam
Étiquette de machine	**marquer un rôle ou une caractéristique ; peut être calculée d'après l'inventaire**	`compute`, `storage`, `gpu`	contrainte `tags=` ; Sunbeam choisit les rôles d'après elles
Étiquette de disque, de réseau	**désigner un périphérique précis**	`ceph` sur un disque	Sunbeam y place les disques de Ceph
Zone	**regrouper par domaine de panne : baie, salle**	`baie-a`, `baie-b`	répartition des unités entre zones
Groupe de ressources	**réserver des machines à une équipe, avec droits**	`cloud-prod`, `labo`	cloisonne plusieurs déploiements
Schéma de stockage	**partitionnement appliqué au déploiement**	plat, LVM, bcache, RAID logiciel	disque système d'un côté, disques de données vierges de l'autre
Images	**systèmes proposés au déploiement, synchronisés depuis Canonical ou importés**	Ubuntu 24.04 ; images maison construites avec Packer	Juju demande la base Ubuntu voulue par le charm
Hôte de machines virtuelles	**MAAS crée des machines virtuelles sur un hôte LXD et les gère comme des serveurs**	contrôleurs, maquettes	machines de gouvernance du cloud

Bonne pratique : décider des étiquettes, zones et espaces **avant** d'enrôler les machines. C'est le vocabulaire que tous les outils au-dessus de MAAS utiliseront pour placer les services.

MAAS en pratique : installation et commandes

Prérequis : une machine Ubuntu Server 24.04 dédiée, une interface sur le réseau d'amorçage, aucun autre serveur DHCP sur ce réseau.

```
# 1 · Installer (maquette : base de test intégrée)
sudo snap install maas --channel=3.7/stable
sudo snap install maas-test-db
sudo maas init region+rack \
  --database-uri maas-test-db:///
sudo maas createadmin

# 2 · Ouvrir une session en ligne de commande
sudo maas apikey --username admin > cle.txt
maas login admin \
  http://10.10.0.2:5240/MAAS/api/2.0/ $(cat cle.txt)

# 3 · Dans l'interface web : importer les images,
#     activer le DHCP sur le VLAN d'amorçage,
#     définir la plage dynamique
```

En production, la base de test est remplacée par un PostgreSQL externe : version 16 recommandée, la 14 étant le minimum, déjà dépréciée.

```
# Inventaire
maas admin machines read | jq '.[].hostname'

# Étiqueter
maas admin tags create name=compute
maas admin tag update-nodes compute add=$ID

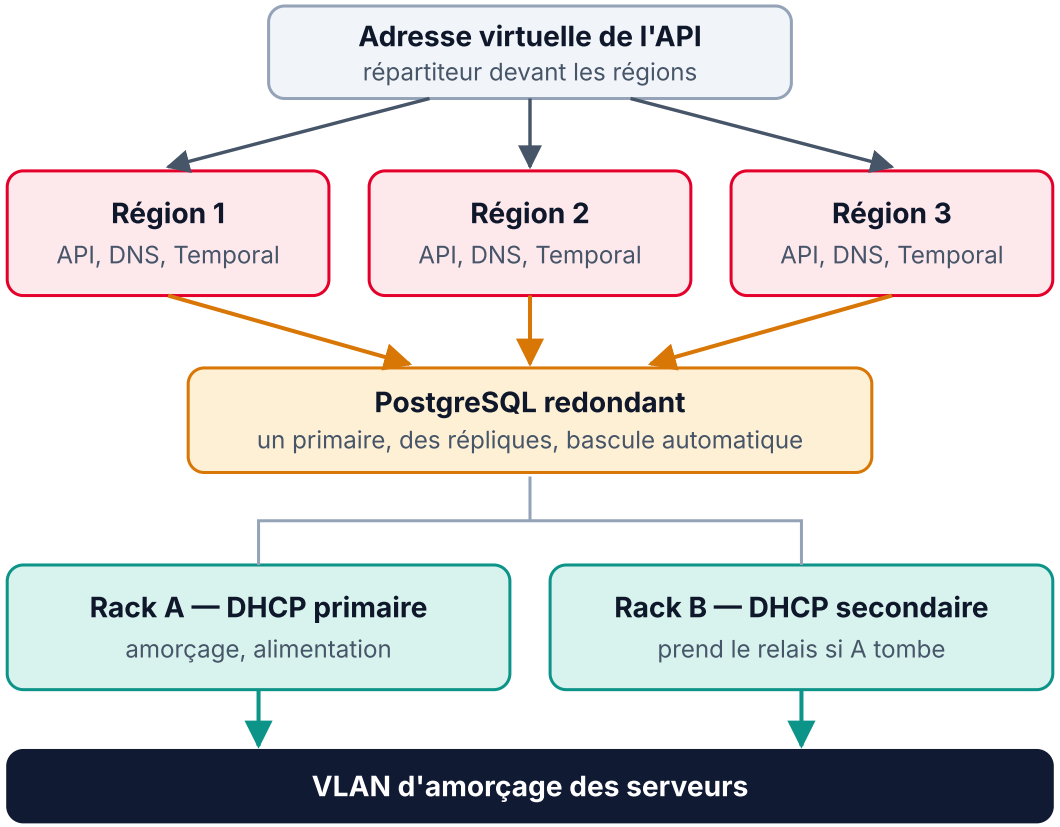
# Relancer l'inventaire d'une machine
maas admin machine commission $ID

# Réserver puis déployer
maas admin machines allocate tags=compute
maas admin machine deploy $ID \
  distro_series=noble

# Rendre au parc en effaçant les disques
maas admin machine release $ID erase=true
```

- `$ID` est l'identifiant système de la machine (`system_id`), visible dans l'inventaire.
- Tout ce que fait l'interface web passe par la même API : ce qui se clique se scripte.

Haute disponibilité et versions



Version	Apports marquants
3.5	Temporal pour les enchaînements, agent en Go, images stockées par la région, déploiement en mémoire, Oracle Linux
3.6	première version native sur Ubuntu 24.04, adresses IP réservées, tableau de bord Grafana
3.7	cartes NVIDIA BlueField-3, ligne de commande plus rapide, résolveur DNS côté rack

- **Ce qui tombe si MAAS tombe :** on ne peut plus enrôler, déployer ni libérer. Les machines **déjà déployées continuent de tourner**.
- **Attention au DNS et au DHCP :** si les machines déployées utilisent MAAS comme résolveur, sa disponibilité compte au quotidien.
- **À sauvegarder :** la base PostgreSQL et les fichiers de configuration du snap. Sans la base, l'inventaire et les adresses attribuées sont perdus.
- **Mise à jour :** changement de canal du snap, une région après l'autre.

MAAS : avantages et limites

Avantages

- **Complet :** découverte, inventaire, tests, réseau, stockage et installation dans un seul outil, avec DHCP et DNS intégrés.
- **Rapide :** une machine passe de « Ready » à « Deployed » en quelques minutes.
- **Piloté par API**, donc par Juju, Terraform, Ansible ou vos scripts.
- **Multi-systèmes :** Ubuntu d'abord, mais aussi d'autres Linux, Windows et ESXi.
- **Indépendant du reste :** utile même sans OpenStack, pour un parc Kubernetes ou des serveurs de calcul.
- **Libre**, sans limite de nombre de machines.

Limites

- **Exige la maîtrise du réseau d'amorçage :** cohabiter avec un DHCP d'entreprise demande un relais et de la rigueur.
- **Dépend de la qualité des cartes de gestion :** micrologiciels anciens ou exotiques sont la première source d'échecs.
- **Un service de plus à rendre redondant** et à sauvegarder, avec sa base PostgreSQL.
- **Images non Ubuntu :** à construire soi-même ; certaines relèvent de l'abonnement.
- **Ne configure pas les commutateurs :** VLAN et agrégats doivent exister côté réseau avant.

Quand l'utiliser

Dès que le parc dépasse une dizaine de machines, ou que l'on veut pouvoir reconstruire un nœud sans intervention manuelle.

Quand s'en passer

Maquette sur une à trois machines, ou parc déjà géré par un autre outil de provisionnement que l'on ne veut pas remplacer.

Choix irréversible dans Sunbeam

Le fournisseur de machines — manuel ou MAAS — se décide à la conception : on ne peut pas en changer une fois le cloud déployé.

03

Juju — orchestrer par charms

Un moteur qui déploie des applications et, surtout, les relie entre elles : le savoir-faire d'exploitation n'est plus dans la tête de l'administrateur mais dans du code réutilisable.

- Rôle et principe
- Le vocabulaire du modèle
- Les relations
- Charmhub
- Juju au-dessus de MAAS
- Architecture
- Anatomie d'un charm
- Charms machine et charms Kubernetes
- Commandes essentielles
- Fonctions avancées, versions, bilan

Juju : rôle et principe

Le problème résolu : installer un logiciel est facile ; le **relier** aux autres, le faire évoluer et le maintenir pendant des années ne l'est pas.

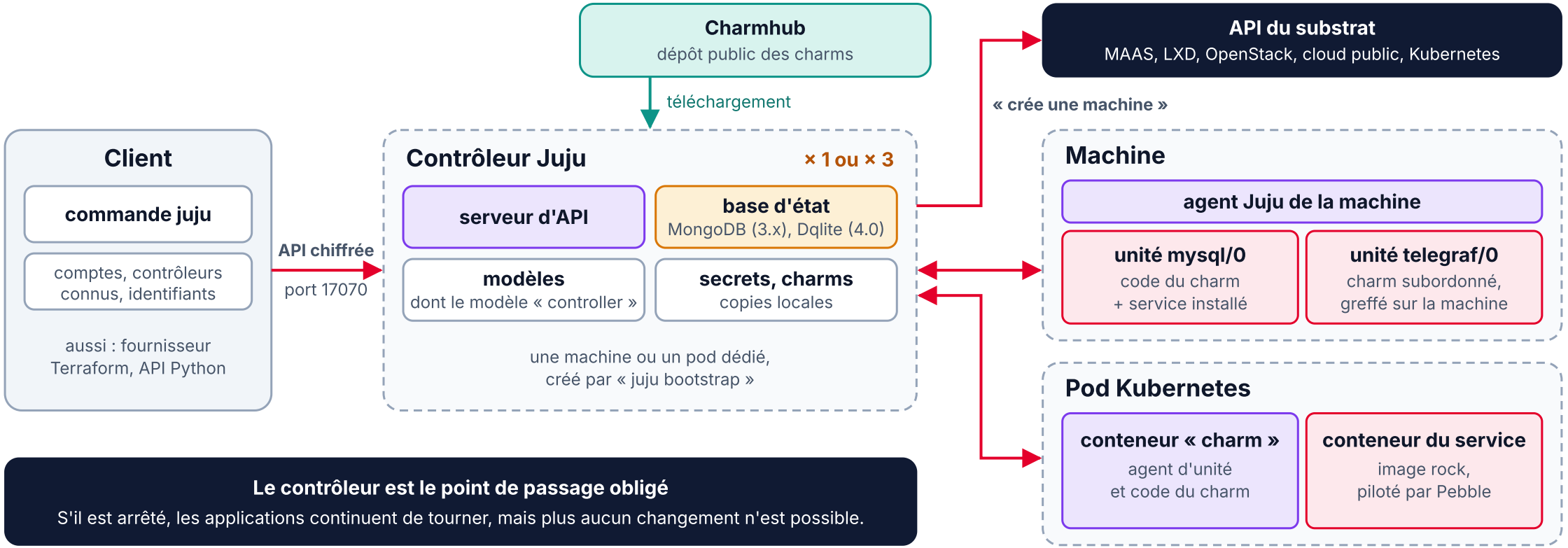
- **Un charm par application :** il contient tout ce qu'un spécialiste saurait faire — installer, configurer, initialiser la base, mettre à jour, sauvegarder.
- **Des relations entre applications :** « Keystone a besoin d'une base » se dit en une commande ; les deux charms échangent adresses et identifiants sans intervention.
- **Un modèle, plusieurs substrats :** la même commande déploie sur MAAS, sur LXD, sur un cloud public ou sur Kubernetes.
- **Tout le cycle de vie :** mise à l'échelle, changement d'option, montée de version, actions ponctuelles.

> **Idée centrale :** Juju ne sait rien d'OpenStack ni de PostgreSQL. Il transmet des **événements** ; c'est le charm qui sait quoi faire.

Question	Ansible	Helm	Juju
Unité de réutilisation	rôle	chart	charm
Nature	tâches exécutées à la demande	gabarits de manifestes	programme réagissant à des événements
Agent permanent	non	non	oui, sur chaque unité
Liaison entre applications	variables à tenir cohérentes	valeurs à tenir cohérentes	relations négociées par les charms
Cibles	machines	Kubernetes	machines et Kubernetes
État connu	non conservé	historique des versions	modèle tenu par le contrôleur

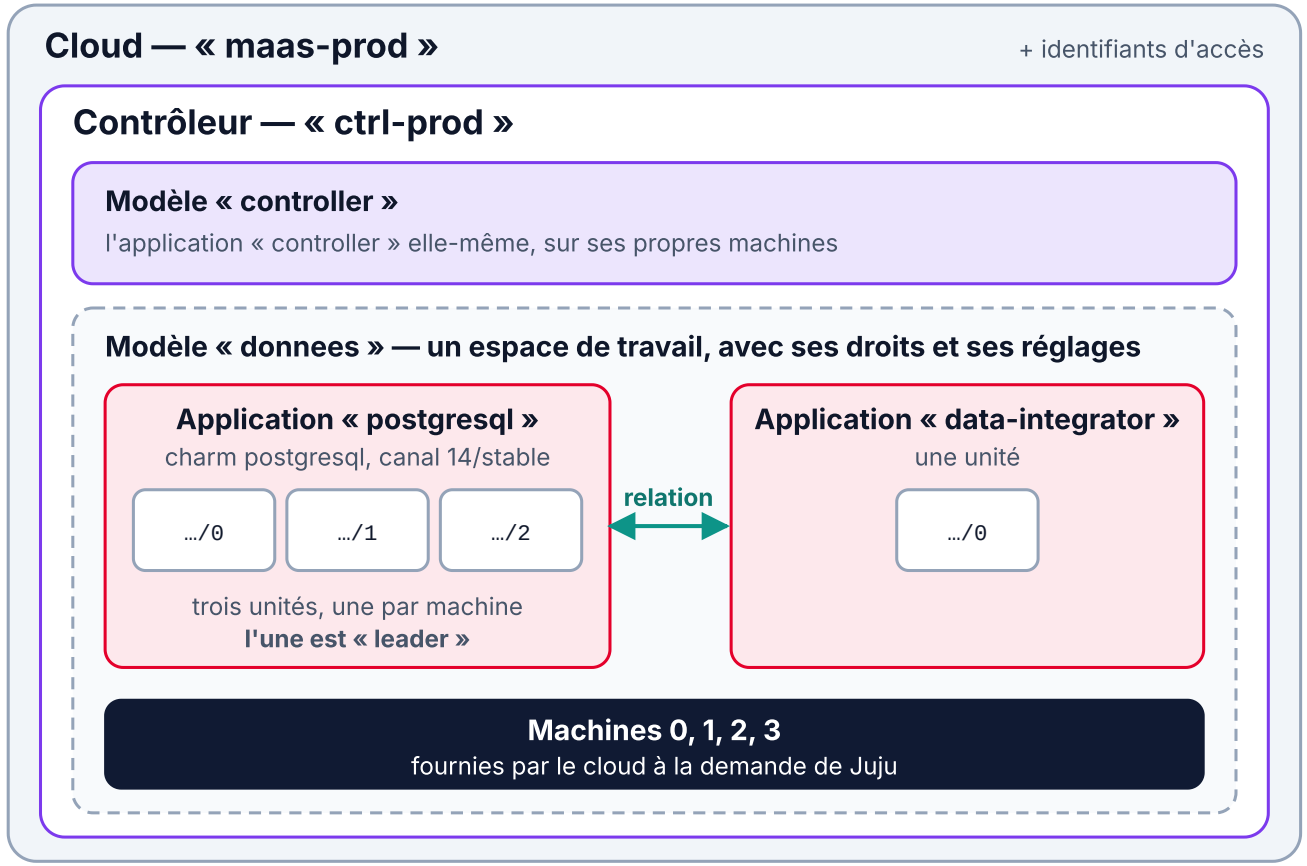
> **Contrepartie :** on dépend de la **qualité du charm**. Ce qu'il ne prévoit pas ne se fait pas proprement ; le contourner à la main sera défait par le prochain événement.

Architecture : client, contrôleur, agents



Les agents ouvrent la connexion vers le contrôleur : ils reçoivent les événements, renvoient l'état.

Le vocabulaire du modèle



Terme	Définition
Cloud	un fournisseur de ressources : MAAS, LXD, Kubernetes…
Contrôleur	le service qui tient l'état et pilote un ou plusieurs modèles
Modèle	un ensemble d'applications gérées ensemble ; un espace de noms sur Kubernetes
Application	un charm déployé, avec sa configuration
Unité	une instance en marche de l'application
Leader	l'unité élue pour les opérations à ne faire qu'une fois
Machine	le serveur ou la machine virtuelle qui porte des unités
Relation	un lien déclaré entre deux applications

> **Lire `juju status`, c'est lire ce schéma :** le modèle, ses applications, leurs unités, les machines, puis les relations.

Anatomie d'un charm

```
# charmcraft.yaml — la carte d'identité
name: demo-api
requires:
  database:
    interface: postgresql_client
provides:
  metrics-endpoint:
    interface: prometheus_scrape
config:
  options:
    port: { type: int, default: 8080 }
actions:
  rotate-token: { description: "Renouvelle le jeton" }
```

```
# src/charm.py — le comportement
class DemoCharm(ops.CharmBase):
  def __init__(self, framework):
    super().__init__(framework)
    framework.observe(self.on.config_changed,
                      self._on_config_changed)

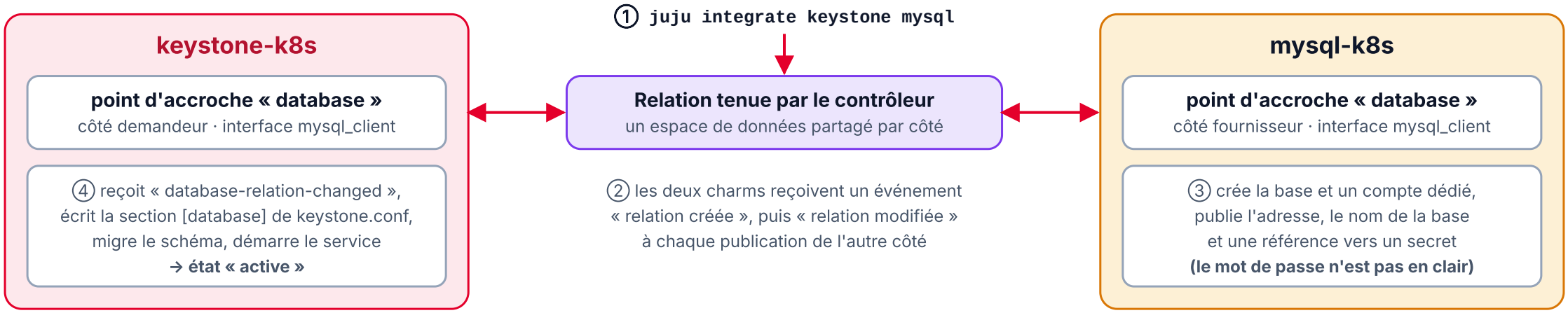
  def _on_config_changed(self, event):
    self._write_config(self.config["port"])
    self._restart_service()
    self.unit.status = ops.ActiveStatus()
```

Événement reçu	Quand
`install`	première arrivée de l'unité
`config-changed`	une option a été modifiée
`start`, `stop`	démarrage, arrêt de l'unité
`<relation>-changed`	l'autre côté a publié des données
`leader-elected`	cette unité devient leader
`upgrade-charm`	nouvelle révision du charm
`update-status`	à intervalle régulier

État affiché	Signification
`active`	le service rend son service
`waiting`	attend une autre application
`blocked`	attend **l'administrateur** : relation ou option manquante
`maintenance`	opération en cours
`error`	un traitement a échoué : lire le journal

> Un charm se développe avec la bibliothèque Python **ops** et se construit avec **charmcraft**.

Les relations : le cœur de Juju



- **Une relation n'est possible qu'entre interfaces identiques :** un demandeur `mysql_client` ne se branche que sur un fournisseur `mysql_client`. C'est un contrat.
- **Elle vit dans le temps :** si l'adresse de la base change ou si une unité s'ajoute, l'autre côté est prévenu et se reconfigure seul.
- **La retirer** (`juju remove-relation`) déclenche le nettoyage : compte supprimé, service reconfiguré.
- **Relation entre modèles :** une application peut être **offerte** à un autre modèle, voire à un autre contrôleur — la supervision ou un Vault partagé, par exemple.
- **Relation « pair » :** entre les unités d'une même application, pour former une grappe.
- **Dans Canonical OpenStack :** plusieurs dizaines de relations sont créées pour vous ; `juju status --relations` les montre toutes.

Charms machine et charms Kubernetes

Charm machine

exemples : microceph, openstack-hypervisor

Une machine Ubuntu (serveur ou machine virtuelle)

agent Juju + code du charm
exécuté à chaque événement

→

service
paquet deb ou snap, unité systemd

le charm agit sur le système lui-même
apt, snap, systemctl, fichiers de configuration, disques, interfaces

Convient à ce qui touche au matériel et au noyau.

Charm Kubernetes

exemples : keystone-k8s, mysql-k8s

Un pod, créé par Juju dans l'espace de noms du modèle

conteneur « charm »
agent d'unité + code du charm

→

conteneur du service
rock ; Pebble lance les processus

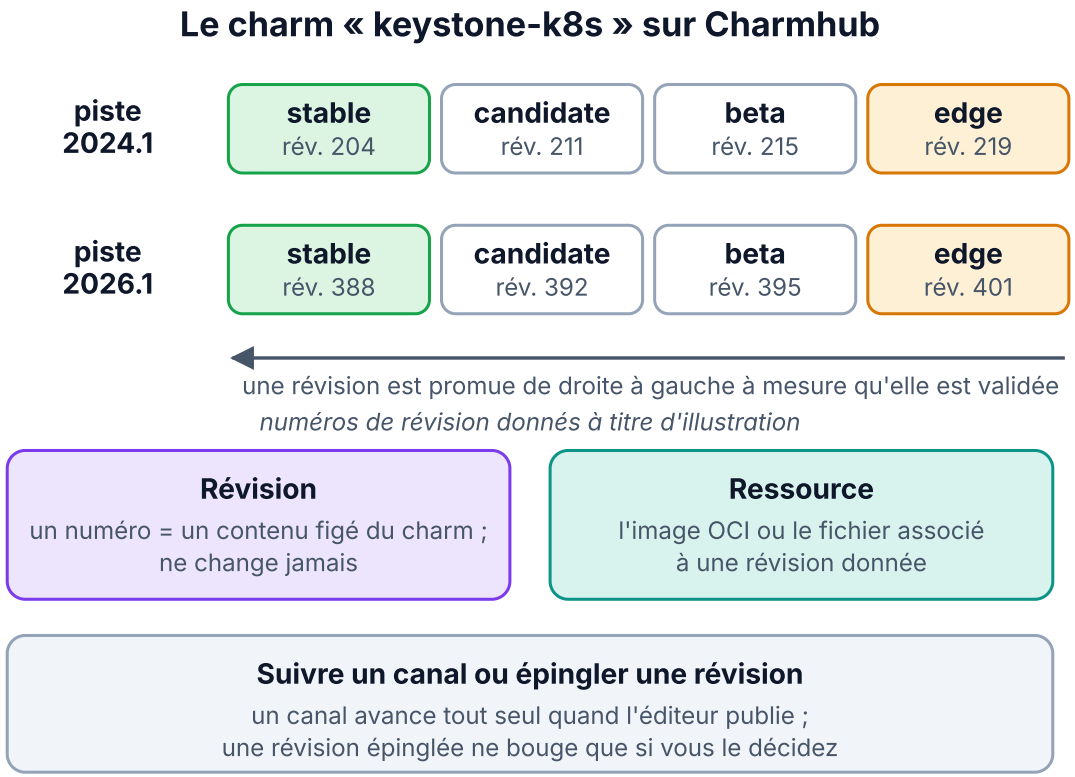
le charm pilote son voisin par l'API de Pebble
pousser un fichier, démarrer ou redémarrer un processus, lire son état

Kubernetes replanifie le pod ; le charm le reconfigure à son retour.

	Charm machine	Charm Kubernetes
Ajouter une instance	`juju add-unit` : une machine de plus	`juju scale-application` : un pod de plus
Livraison du service	deb ou snap installé par le charm	image OCI attachée au charm
Stockage	disques de la machine	volumes persistants
Exposition	adresse de la machine	service Kubernetes, entrée (Traefik)

Les deux se relient entre eux : dans Canonical OpenStack, les charms du plan de contrôle (Kubernetes) et ceux de l'hyperviseur et de Ceph (machines) vivent dans deux modèles, reliés par des relations entre modèles.

Charmhub : pistes, canaux, révisions



- **Piste** (*track*) : une ligne de versions du logiciel. Pour les charms d'OpenStack, la piste porte le nom de la version : `2024.1`, `2026.1`.
- **Risque :** `stable` pour la production, `candidate` et `beta` pour valider, `edge` pour le dernier état du développement.
- **Canal** = piste + risque : `2026.1/stable`.
- `juju refresh` fait passer une application à la dernière révision de son canal, ou vers un autre canal.

Besoin	Commande
Voir les canaux d'un charm	`juju info keystone-k8s`
Déployer depuis un canal	`juju deploy … --channel 2026.1/stable`
Mettre à jour	`juju refresh keystone`
Changer de canal	`juju refresh keystone --channel …`

Réseau isolé : Charmhub est un service en ligne. Sans accès, il faut un mandataire filtrant ou un dépôt interne — à prévoir dès la conception.

Juju en pratique : les commandes essentielles

Prérequis : un poste Ubuntu, un cloud joignable (ici MAAS) et sa clé d'API.

```
# maas-cloud.yaml
clouds:
  maas-prod:
    type: maas
    auth-types: [oauth1]
    endpoint: http://10.10.0.2:5240/MAAS
```

```
# 1 · Installer le client et déclarer le cloud
sudo snap install juju
juju add-cloud --client maas-prod ./maas-cloud.yaml
juju add-credential maas-prod   # colle la clé d'API

# 2 · Créer le contrôleur sur une machine étiquetée
juju bootstrap maas-prod ctrl-prod \
  --constraints tags=juju-controller

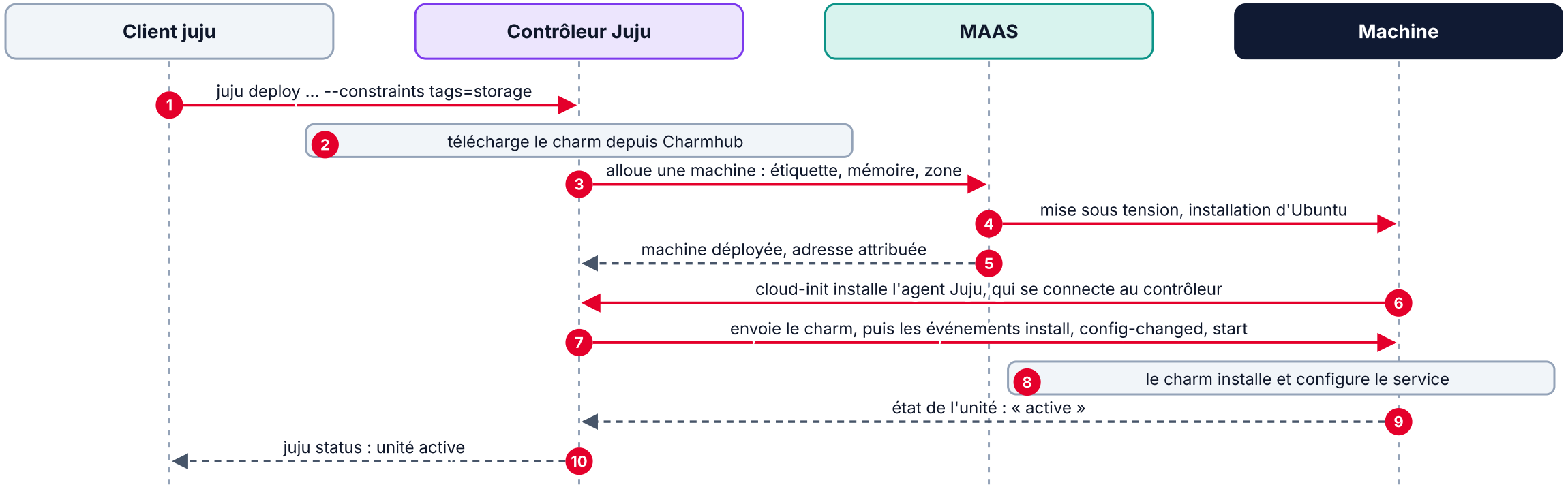
# 3 · Créer un modèle et déployer
juju add-model donnees
juju deploy postgresql --channel 14/stable -n 3
juju deploy data-integrator --config database-name=demo
juju integrate data-integrator postgresql
```

Besoin	Commande
État du modèle	`juju status --relations`
Journal de tous les agents	`juju debug-log`
Lire ou changer une option	`juju config postgresql profile=production`
Ajouter des unités	`juju add-unit postgresql -n 2`
Lancer une action	`juju run postgresql/leader get-password`
Entrer sur une unité	`juju ssh postgresql/0`
Mettre à jour le charm	`juju refresh postgresql`
Changer de modèle	`juju switch donnees`
Retirer une application	`juju remove-application data-integrator`

Réflexe de diagnostic : `juju status` d'abord — une unité `blocked` dit ce qu'elle attend dans son message ; puis `juju debug-log --include <unité>` pour le détail.

Ne jamais corriger à la main un fichier géré par un charm : la valeur sera réécrite au prochain événement. La bonne voie est `juju config`.

Juju au-dessus de MAAS : ce que déclenche « juju deploy »



Les contraintes font le lien : `tags=` , `mem=` , `cores=` , `zones=` , `spaces=` sont traduites en critères de sélection dans l'inventaire de MAAS.

Retirer l'unité rend la machine : `juju remove-unit` puis `juju remove-machine` la font repasser à l'état « Ready » dans MAAS, disques effacés si la politique le prévoit.

Fonctions avancées utiles en exploitation

Fonction	À quoi elle sert	Exemple
Contraintes	**choisir le type de machine pour une application**	`juju deploy ceph-osd --constraints "tags=storage mem=64G"`
Espaces réseau	**attacher chaque point d'accroche d'un charm au bon réseau**	`juju deploy mysql --bind "database=interne"`
Stockage	**demander des volumes au substrat pour une unité**	`juju deploy postgresql --storage pgdata=100G`
Actions	**opérations ponctuelles prévues par le charm : sauvegarde, rotation, bascule**	`juju run mysql/leader create-backup`
Secrets	**mots de passe et clés gérés par le contrôleur, partagés par relation, renouvelables**	`juju add-secret`, `juju grant-secret`
Offres entre modèles	**partager une application avec d'autres modèles ou contrôleurs**	`juju offer prometheus:metrics-endpoint`
Migration de modèle	**déplacer un modèle vers un autre contrôleur, sans arrêter les applications**	`juju migrate donnees ctrl-neuf`
Contrôleur redondant	**trois instances du contrôleur, avec réplication de la base d'état**	`juju enable-ha` en 3.x ; ajout d'unités au contrôleur en 4.0

Automatisation : le fournisseur Terraform pour Juju décrit modèles, applications et relations dans des fichiers versionnés. C'est ainsi que Sunbeam pilote Juju en interne.

À sauvegarder : le contrôleur (`juju create-backup` en 3.x) et les identifiants du client. Perdre le contrôleur sans sauvegarde, c'est perdre la main sur tout ce qu'il gère.

Juju 3.6 LTS et Juju 4.0 : où en est-on ?

	Juju 3.6	Juju 4.0
Statut	**version à long terme**	version majeure, sortie le 14 novembre 2025
Base d'état	**MongoDB**	Dqlite : SQLite répliqué par Raft
Contrôleur redondant	`juju enable-ha`	`juju add-unit` sur le modèle « controller »
Lots de charms	**encore gérés**	`export-bundle` retiré ; Terraform conseillé
Fournisseur « manual »	**« manual »**	renommé « unmanaged »
Clés SSH	**ajoutées aux nouveaux modèles**	plus ajoutées automatiquement

Retiré en 4.0

- Les « séries » (remplacées par les bases), les charms Kubernetes d'ancienne génération, le fournisseur KVM.
- `juju wait-for`, l'option `--watch` de `juju status`, les réglages de leader.

Politique de versions

- Une version mineure environ **tous les trois mois**, des correctifs mensuels.
- Une version mineure reçoit des correctifs pendant quatre mois, puis deux mois de correctifs de sécurité seulement.
- À chaque version majeure, la dernière mineure de la série précédente devient **version à long terme**, avec des correctifs de sécurité annoncés sur quinze ans.

Passer de 3.6 à 4.0

- Pas de mise à jour en place du contrôleur : on **crée un contrôleur 4.0** et on y **migre les modèles**.
- Un guide « 3.6 vers 4.0 » est publié (4.0.15 en octobre 2026). Il liste surtout les ruptures à traiter avant : `enable-ha` et `upgrade-machine` retirés, espaces MAAS explicites. À dérouler d'abord en maquette.

> **Pour Canonical OpenStack :** la version de Juju est celle qu'installe Sunbeam. Ne pas la remplacer de sa propre initiative ; suivre la documentation de la version du cloud.

Juju : avantages et limites

Avantages

- **Le savoir-faire est dans le charm :** une petite équipe exploite des logiciels complexes sans en maîtriser chaque détail.
- **Les relations** suppriment toute une classe d'erreurs : adresses, comptes et secrets ne sont plus recopiés à la main.
- **Un seul outil** pour les machines et pour Kubernetes, et pour tout le cycle de vie.
- **État lisible :** `juju status` donne une vue complète et homogène, quelle que soit l'application.
- **Catalogue** de charms maintenus par Canonical : bases de données, supervision, identité, stockage.

Limites

- **Boîte noire relative :** comprendre pourquoi un charm est bloqué demande parfois de lire son code.
- **Options limitées à ce que le charm expose :** un réglage non prévu ne se fait pas proprement.
- **Compétence rare** sur le marché, comparée à Ansible ou Kubernetes.
- **Le contrôleur est critique :** à rendre redondant, à sauvegarder, à mettre à jour avec soin.
- **Écosystème centré sur Ubuntu :** peu de charms pour d'autres systèmes.
- **Transition 3.x vers 4.0** à planifier : changement de base d'état, commandes retirées.

À retenir sur le modèle

Contrôleur, modèle, application, unité, relation : cinq mots suffisent à lire n'importe quel déploiement.

À retenir sur les charms

Un charm réagit à des événements. Son état — `active`, `waiting`, `blocked` — dit où regarder.

À retenir pour la suite

Canonical OpenStack est un ensemble de charms. Savoir lire Juju, c'est savoir diagnostiquer le cloud.

04

Canonical OpenStack (Sunbeam)

La distribution OpenStack de Canonical : un plan de contrôle en pods Kubernetes, des snaps sur les hôtes, le tout piloté par Juju derrière une seule commande.

- Positionnement
- Rôles des machines
- Quatre architectures de référence
- Installation manuelle, puis avec MAAS
- Exploitation et dimensionnement
- Architecture et artefacts
- Ce qui tourne où
- Réseaux
- Manifeste et fonctionnalités
- Sécurité, disponibilité, topologies

De Charmed OpenStack à Sunbeam

	Canonical OpenStack (Sunbeam)	Charmed OpenStack
Génération	**actuelle, recommandée**	précédente, maintenue
Plan de contrôle	**pods sur Canonical Kubernetes**	conteneurs LXD sur des machines
Charms	**Kubernetes, plus quelques charms machine**	charms machine
Pilotage	**commande `sunbeam`**	Juju directement, lots de charms
Machines nues	**manuel ou MAAS**	MAAS
Stockage	**MicroCeph**	Ceph par charms
Réseau virtuel	**OVN**	OVN
Taille minimale	**une machine**	plusieurs machines
Dernière version	**2026.1 LTS**	2024.1, sur Ubuntu 22.04

Sunbeam est aussi un projet officiel d'OpenStack, sous la gouvernance de l'Open Infrastructure Foundation : le code est ouvert et développé en amont.

Pourquoi Canonical a changé de génération

- **Barrière d'entrée :** Charmed OpenStack exigeait de maîtriser MAAS, Juju et des dizaines de charms avant le premier résultat.
- **Mises à jour :** un plan de contrôle en pods se met à jour progressivement, service par service.
- **Isolement :** les services ne dépendent plus de la version du système hôte.
- **Échelle basse :** pouvoir démarrer sur une seule machine, puis grandir sans réinstaller.

Ce que fait la commande `sunbeam`

- Prépare les machines et met le cluster en grappe.
- Installe Kubernetes, crée le contrôleur Juju et les modèles.
- Applique des plans Terraform qui déploient et relient les charms.
- Pose les questions de configuration, ou lit un **manifeste**.
- Active les fonctionnalités optionnelles, met à jour, ajoute et retire des nœuds.

On n'a pas à manipuler Juju pour installer — mais on s'en sert pour **observer et diagnostiquer**.

Architecture : quatre fonctions, trois types d'artefacts

Gouvernance

déployer, opérer, observer

- **Sunbeam** — client et service de grappe
- **contrôleur Juju** — modèles, charms, relations
- **MAAS** — si fournisseur MAAS
- **supervision** — optionnelle

Contrôle

API, ordonnanceurs, conducteurs

- **services OpenStack** — Keystone, Nova, Neutron, Glance…
- **MySQL, RabbitMQ** — état et messages
- **Traefik** — points d'accès publics et internes
- **bases OVN** — nord et sud

Calcul et réseau

faire tourner les instances

- **nova-compute** — libvirt, QEMU, KVM
- **OVN et Open vSwitch** — réseau virtuel sur chaque hôte
- **service de métadonnées** — pour les instances
- **passerelles OVN** — nœuds « network » (MicroOVN)

Stockage

conserver les données

- **MicroCeph** — moniteurs, gestionnaires, OSD
- **cinder-volume** — pilote les volumes
- **passerelle objet** — optionnelle

Ceph par défaut ; baies Pure Storage, Dell et Hitachi (option) en plus

Rocks

images OCI : services du plan de contrôle et pile de supervision

→ **fonction Contrôle**

Snaps

applications confinées : processus du plan de données et beaucoup d'autres composants

→ **Calcul, Réseau, Stockage, Gouvernance**

Charms

opérateurs qui déploient, configurent, mettent à l'échelle et relient rocks et snaps

→ **partout**

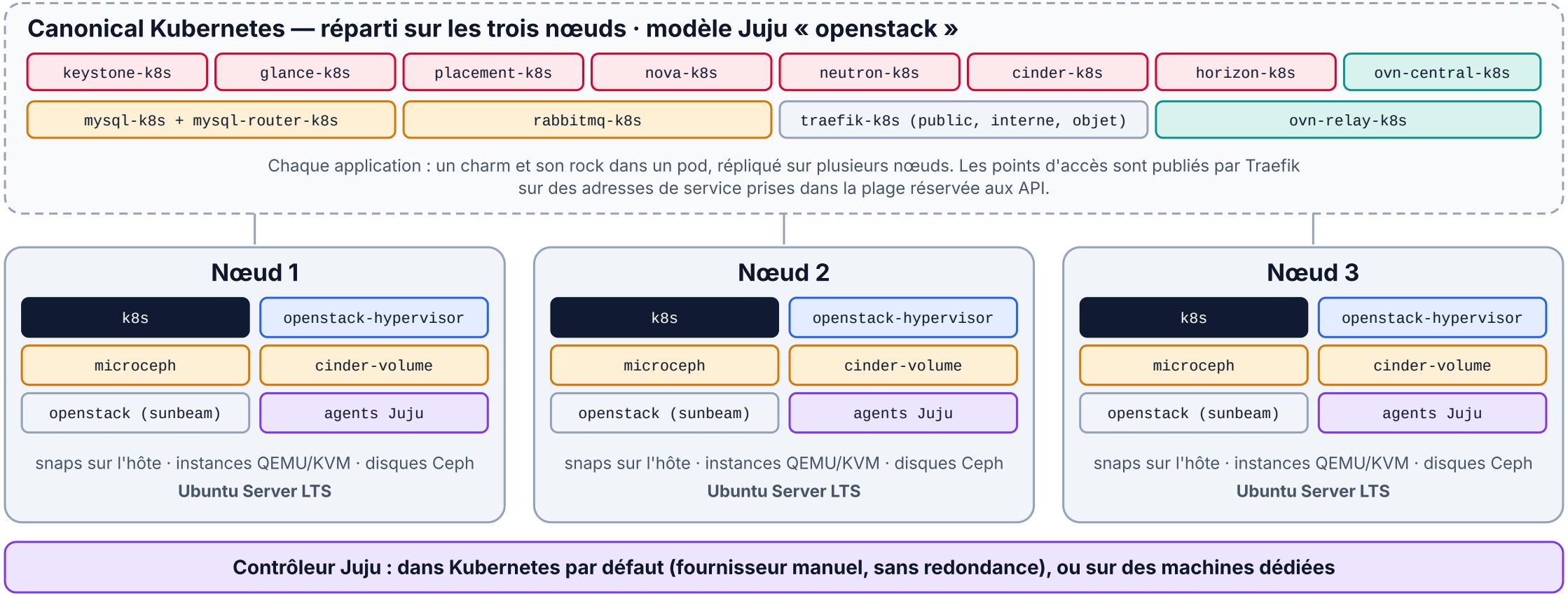
Les rôles des machines

Rôle	Ce que porte la machine	Étiquette MAAS	Remarques
Control	**Canonical Kubernetes et les charms du plan de contrôle : API, base, bus de messages**	`control`	trois pour la haute disponibilité
Compute	**nova-compute, libvirt et QEMU, OVN et Open vSwitch, métadonnées**	`compute`	matériel compatible KVM recommandé
Network	**MicroOVN : passerelles de niveau 3 d'OVN**	`network`	**incompatible avec Compute** sur la même machine
Storage	**MicroCeph**	`storage`	disques vierges, non partitionnés ; étiquette `ceph` sur chacun
Cloud	**cumul Control, Compute et Storage sur la même machine**	les trois	le nœud type d'un petit cloud
Sunbeam Controller	**service de grappe de Sunbeam**	`sunbeam`	avec MAAS : machine virtuelle dédiée
Juju Controller	**contrôleur Juju**	`juju-controller`	hors des machines de charge en multi-nœud
Sunbeam Client	**le poste d'où l'on lance les commandes**	—	une machine virtuelle suffit
Governor	**machine physique qui héberge MAAS, les contrôleurs et le client en machines virtuelles**	—	installée à la main, hors MAAS

Avec le fournisseur manuel, les rôles se donnent à la commande : `--role control,compute,storage` . **Avec MAAS**, ils se lisent sur les étiquettes des machines, toutes marquées en plus `openstack-<nom du déploiement>` .

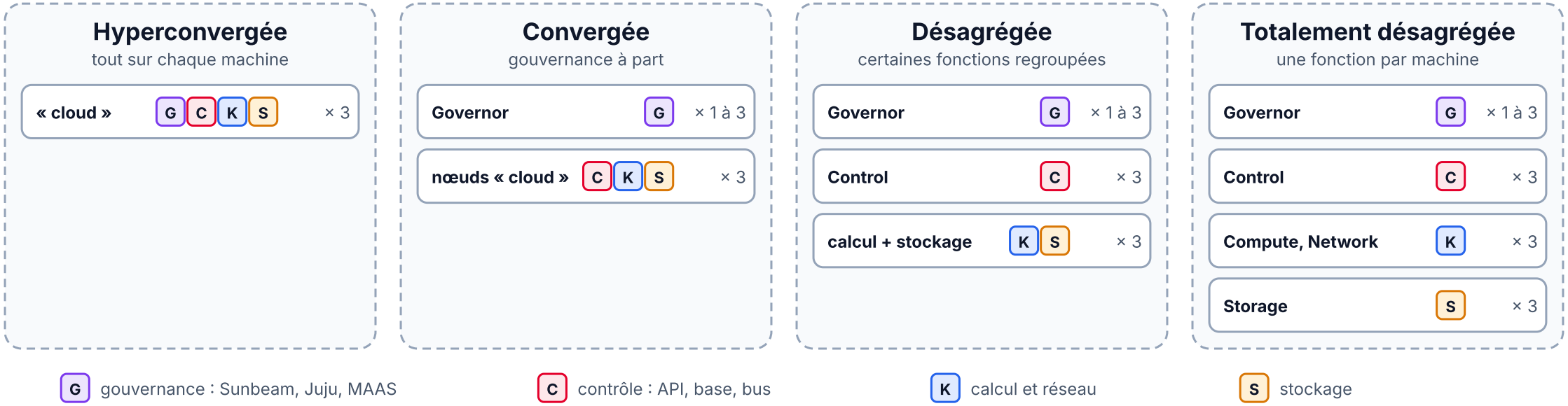
Évolution annoncée : les passerelles OVN tournent encore sur les nœuds de calcul ; la documentation prévoit qu'elles ne seront plus fournies, à terme, que par les nœuds « network ».

Ce qui tourne où : un cluster de trois nœuds « cloud »



Deux modèles Juju à connaître : « openstack » pour les charms Kubernetes du plan de contrôle, et « admin/controller » pour les charms qui pilotent les snaps des hôtes. Premières commandes de diagnostic : `juju status -m openstack` , puis `juju status -m admin/controller` .

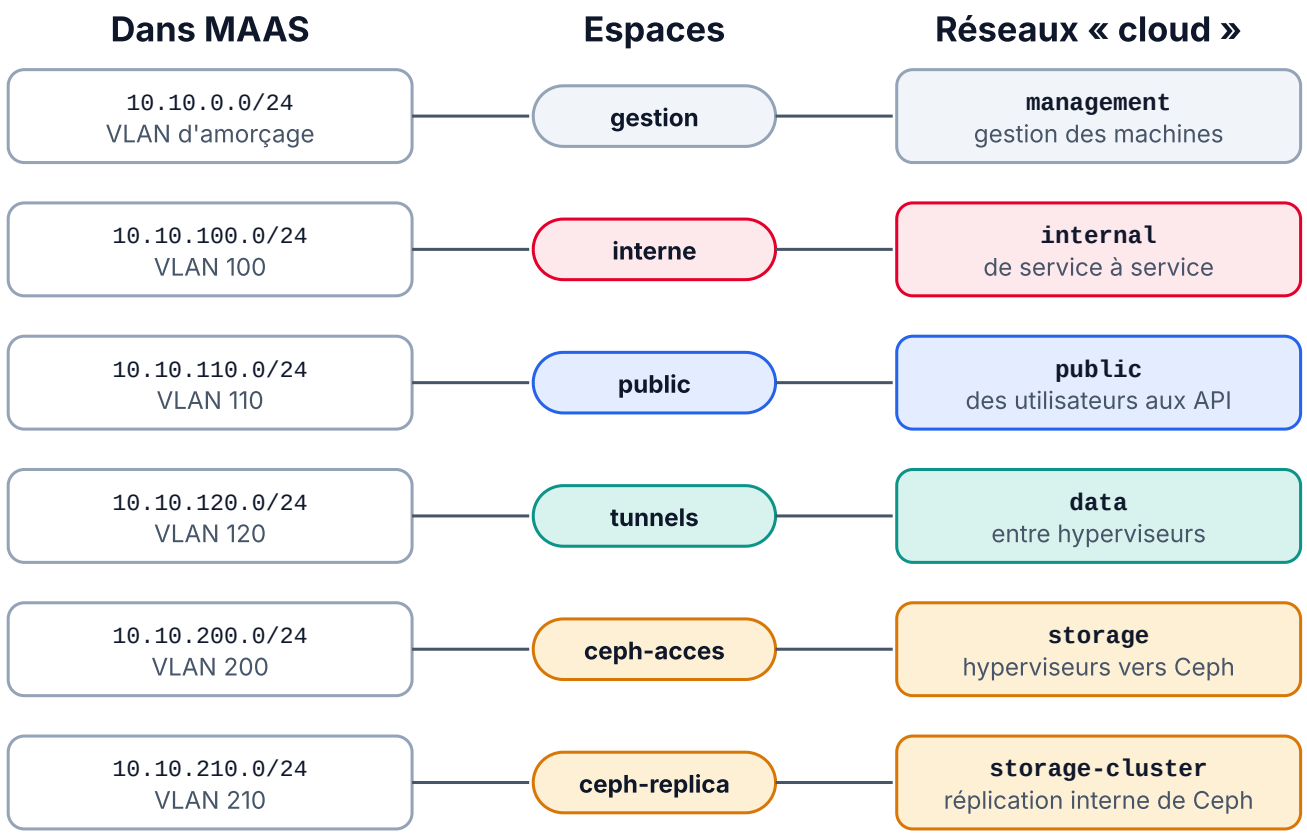
Quatre architectures de référence



Taille	Réseau physique	Architecture	Machines nues
1 à 9 machines	un seul niveau	hyperconvergée	fournisseur manuel
10 machines et plus	deux niveaux, *spine-leaf*	convergée, désagrégée ou totalement désagrégée	MAAS

Haute disponibilité : facultative mais recommandée ; elle exige **au moins trois machines de chaque type**, quelle que soit l'architecture retenue. Recommandations issues de la documentation de conception de Canonical.

Réseaux : deux réseaux physiques, six réseaux « cloud »



Adressage et noms d'espaces donnés en exemple.

Le minimum : deux réseaux physiques

- **Générique :** provisionnement, gestion et accès aux API.
- **Externe :** adresses flottantes et sortie des instances vers l'extérieur.

Avec MAAS : une isolation plus fine

- Chaque sous-réseau est rangé dans un **espace** ; chaque espace est associé à un **réseau « cloud »** par `sunbeam deployment space map`.
- Un seul espace pour tout : aucune isolation. Six espaces : séparation complète.
- **Plages d'adresses à réserver** dans MAAS, nommées `<nom>-internal-api` et `<nom>-public-api` : dix à vingt adresses chacune, pour les points d'accès.

Réseau externe : sur l'interface qui y est branchée, laisser le sous-réseau « non configuré » dans MAAS et poser l'étiquette réseau `neutron:physnet1`. C'est OVN qui s'en servira, pas le système hôte.

Installation avec le fournisseur manuel

Prérequis, avant toute commande

- Une machine physique dédiée, **Ubuntu Server 24.04 LTS** fraîchement installé, architecture amd64.
- **Deux réseaux** physiques avec accès à Internet (ou un mandataire).
- Un nom de machine pleinement qualifié qui se résout.
- Pour le stockage : au moins un disque **vierge**, sans partition.
- Une plage d'adresses libre pour les points d'accès.

```
# Première machine
snap info openstack                  # canaux disponibles
sudo snap install openstack --channel 2026.1/stable
sunbeam prepare-node-script --bootstrap \
  | bash -x && newgrp snap_daemon
sunbeam cluster bootstrap \
  --role control,compute,storage
sunbeam configure --openrc demo-openrc
```

Succès du `bootstrap` : le message « Node has been bootstrapped with roles: storage, compute, control ».

```
# Ajouter une machine — sur la première :
sunbeam cluster add cloud-2.exemple.lan \
  --output cloud-2.asc
# copier cloud-2.asc sur la nouvelle machine, puis, sur celle-ci :
sudo snap install openstack --channel 2026.1/stable
sunbeam prepare-node-script | bash -x \
  && newgrp snap_daemon
cat cloud-2.asc | sunbeam cluster join \
  --role control,compute,storage -

# Après l'ajout de nœuds « control » :
sunbeam cluster resize
```

Étape	Ce qu'elle fait
`prepare-node-script`	dépendances, client Juju, groupe, répertoires
`cluster bootstrap`	Kubernetes, contrôleur Juju, plan de contrôle, hyperviseur, Ceph
`configure`	réseau externe, projet et utilisateur de démonstration
`cluster resize`	étend le plan de contrôle aux nouveaux nœuds

Le contrôleur Juju par défaut, logé dans Kubernetes, **n'est pas redondant**. Pour un cloud qui doit l'être, fournir un contrôleur Juju externe et redondant **avant** le bootstrap.

Installation avec MAAS : ce qu'il faut préparer

Machines

Élément	Nombre	Nature
Governor	1, ou 3	physique, Ubuntu 24.04, installé à la main
MAAS	1, ou 3	machine virtuelle sur un Governor
Sunbeam Client	1	machine virtuelle sur un Governor
Sunbeam Controller	1, ou 3	machine virtuelle, gérée par MAAS
Juju Controller	1, ou 3	machine virtuelle, gérée par MAAS
Nœuds du cloud	3 par rôle	physiques, gérés par MAAS

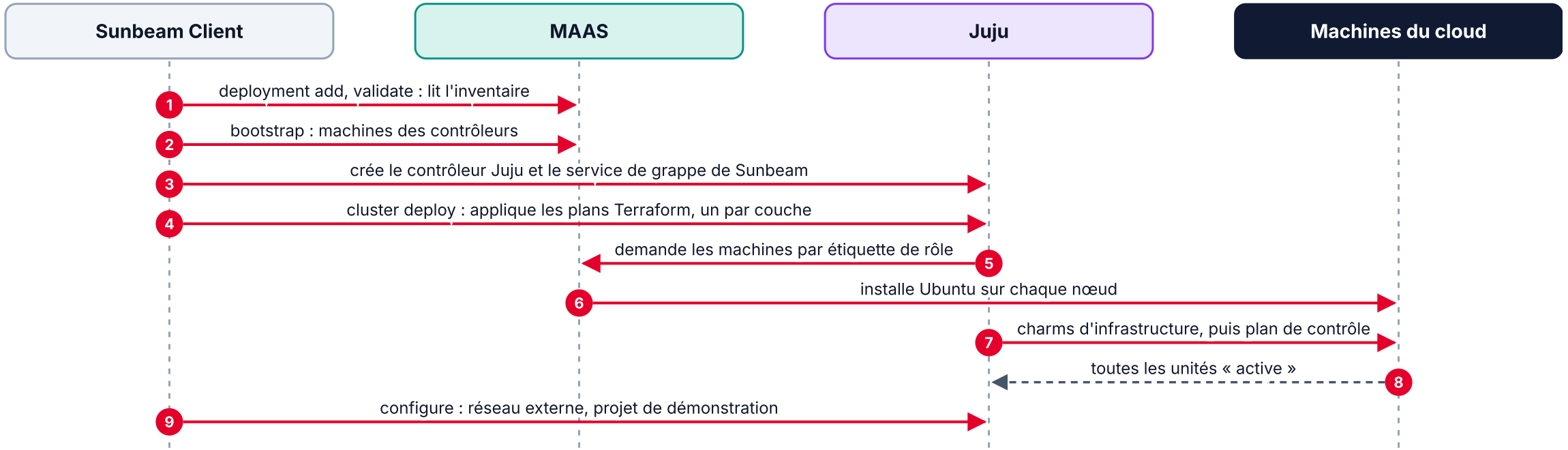
« Ou 3 » : la valeur à retenir pour une haute disponibilité complète. Toutes les machines sauf les Governor et le client doivent être **enrôlées, inventoriées, configurées et étiquetées** dans MAAS.

Dans MAAS, pour un déploiement nommé « prod »

Objet	Valeur attendue
Étiquette commune	`openstack-prod` sur toutes les machines du cloud et les contrôleurs
Étiquettes de rôle	`control`, `compute`, `storage`, `network`, `sunbeam`, `juju-controller`
Étiquette de disque	`ceph` sur chaque disque destiné à Ceph
Étiquette de réseau	`neutron:physnet1` sur l'interface externe, sous-réseau non configuré
Plages réservées	`prod-internal-api`, `prod-public-api` : 10 à 20 adresses
Espaces	un par usage, associés aux sous-réseaux
Clé d'API	celle d'un compte MAAS, pour `sunbeam deployment add`

Les noms sont une convention stricte : le suffixe de l'étiquette et le préfixe des plages doivent reprendre **exactement** le nom du déploiement donné à Sunbeam.

Installation avec MAAS : l'enchaînement




```
# Depuis la machine « Sunbeam Client »
sudo snap install openstack --channel 2026.1/stable
sunbeam prepare-node-script --client | bash -x
sunbeam deployment add maas prod $CLE_API http://10.10.0.2:5240/MAAS
sunbeam deployment space map interne:internal   # un par réseau
sunbeam deployment validate
```

```
sunbeam cluster bootstrap   # couche d'orchestration
sunbeam cluster deploy      # le cloud lui-même
sunbeam configure --openrc demo-openrc

# Plus tard, après avoir étiqueté de nouvelles machines :
sunbeam cluster deploy
```

Le manifeste : décrire le cloud dans un fichier

```yaml
core:
  config:
    bootstrap:
      management_cidr: 10.10.0.0/24
    database: single            # ou multi
    region: RegionOne
    k8s-addons:
      loadbalancer: 10.10.0.100-10.10.0.120
    external-networks:
      physnet1:
        cidr: 192.0.2.0/24
        gateway: 192.0.2.1
        range: 192.0.2.10-192.0.2.50
        network_type: flat
    microceph_config:
      cloud-1.exemple.lan:
        osd_devices: /dev/sdb,/dev/sdc
  software:
    charms:
      keystone-k8s:
        channel: 2026.1/stable
features:
  loadbalancer:
    config:
      amphora_enabled: false
```

Section	Ce qu'elle porte
`core.config`	les réponses aux questions de l'installation : réseaux, région, base, utilisateur de démonstration, disques de Ceph, noms des points d'accès, mandataire
`core.software`	versions : arguments de Juju, canal ou révision et options de chaque charm, source des plans Terraform
`features`	réglages des fonctionnalités optionnelles : supervision, équilibrage de charge, TLS
`storage`	baies tierces (Pure Storage ; Dell, Hitachi en option), par type puis par instance

- **Intérêt :** une installation **sans question**, reproductible, que l'on versionne dans git et que l'on relit.
- **Usage :** le manifeste se passe aux commandes d'installation et de mise à jour par l'option `--manifest`.
- **Point d'ajustement fin :** c'est ici que l'on épingle la révision d'un charm ou que l'on règle une option qu'aucune question ne propose.

> Extrait simplifié, à valeurs d'exemple : la liste complète des clés est dans la référence du manifeste de la documentation.

Fonctionnalités : le socle et les options

Installé d'office

Fonction	Service
Identité	Keystone
Images	Glance
Calcul	Nova, Placement
Réseau	Neutron avec OVN
Stockage bloc	Cinder sur Ceph
Tableau de bord	Horizon

```
sunbeam enable orchestration
sunbeam enable loadbalancer
sunbeam enable vault
sunbeam disable orchestration
```

> Chaque option est un plan Terraform de plus : des charms déployés et reliés au socle, que l'on peut retirer.

À activer selon les besoins

Fonctionnalité	Service sous-jacent
Orchestration	Heat
Équilibrage de charge	Octavia
DNS	Designate
Secrets	Barbican, avec Vault
Conteneurs	Magnum
Systèmes de fichiers partagés	Manila
Stockage objet	passerelle objet de Ceph
Serveurs nus	Ironic
Télémétrie	Ceilometer, Gnocchi, Aodh
Reprise d'instances	Masakari
Optimisation des ressources	Watcher
Supervision	pile d'observabilité Canonical
Validation	Tempest
TLS, LDAP, synchronisation d'images, Ubuntu Pro	intégrations

Exploitation : les opérations du quotidien

Besoin	Commande
État des nœuds	`sunbeam cluster list`
État des services	`juju status -m openstack`
État des machines	`juju status -m admin/controller`
Journal des charms	`juju debug-log -m openstack`
Pods du plan de contrôle	`sudo k8s kubectl get pods -n openstack`
Services d'un hôte	`sudo snap logs openstack-hypervisor`
État de Ceph	`sudo microceph status`
Identifiants d'administration	`sunbeam openrc > admin-openrc`
Adresse du tableau de bord	`sunbeam dashboard-url`
Retirer un nœud	`sunbeam cluster remove <nom>`

Trois niveaux de lecture : Sunbeam pour les nœuds, Juju pour les applications, Kubernetes ou snap pour les processus.

Mettre à jour, dans cet ordre

```
sudo snap refresh openstack    # d'abord le snap
sunbeam cluster refresh k8s    # 1 · Kubernetes
sunbeam cluster refresh vault  # 2 · Vault, s'il est activé
sunbeam cluster refresh mysql  # 3 · bases de données
sunbeam cluster refresh        # 4 · le reste des charms
```

- Rafraîchir le snap **ne met pas à jour** le cloud : ce sont les commandes `cluster refresh` qui le font.
- Fournisseur manuel : rafraîchir le snap sur **tous** les nœuds ; avec MAAS, sur le client seulement.
- Vault reste **scellé** après sa mise à jour : à desceller à la main.

Limite actuelle, écrite dans la documentation : le rafraîchissement **d'une piste à l'autre** — de 2024.1 à 2025.1, par exemple — n'est pas pris en charge par cette procédure, pas plus que le changement de version mineure de Kubernetes. On met à jour **à l'intérieur** d'une piste.

Dimensionnement : les minimums documentés

	Nœud unique	Multi-nœud, par machine
Nombre de machines	**1**	3 au minimum
Processeur	**4 cœurs, amd64**	16 cœurs, amd64
Mémoire	**16 Gio**	32 Gio
Disque système	**100 Gio libres, SSD**	500 Gio libres, SSD
Disque de stockage	**un SSD de 100 Gio, facultatif**	un SSD de 500 Gio, requis
Réseau de contrôle	**1 Gbit/s**	1 Gbit/s, trafic est-ouest
Réseau externe	**1 Gbit/s, facultatif**	1 Gbit/s, trafic nord-sud

Nœud unique : aucune tolérance aux pannes et des performances limitées. Parfait pour apprendre, inadapté à la production.

Ce sont des **minimums pour démarrer**. Un cloud réel se dimensionne d'après les instances à héberger : cœurs, mémoire et capacité Ceph utile après réplication.

Mémoire à réserver par rôle, hors instances

Rôle	Mémoire minimale
Control	Kubernetes 4 Gio + plan de contrôle 10 Gio
Compute	1 Gio pour les agents
Storage	5 Gio par OSD + 2 Gio moniteur et gestionnaire + 2 Gio passerelle objet
Cloud	la somme : 4 + 11 + Ceph
Sunbeam Controller	4 Gio
Juju Controller	4 Gio
Sunbeam Client	1 Gio

- **Exemple de calcul :** un nœud « cloud » avec deux disques Ceph réserve 4 + 11 + 2×5 + 2 + 2 = **29 Gio** avant la première instance.
- **Disques NVMe :** prévoir davantage de mémoire et de cœurs par OSD.
- **Architecture :** amd64 uniquement.

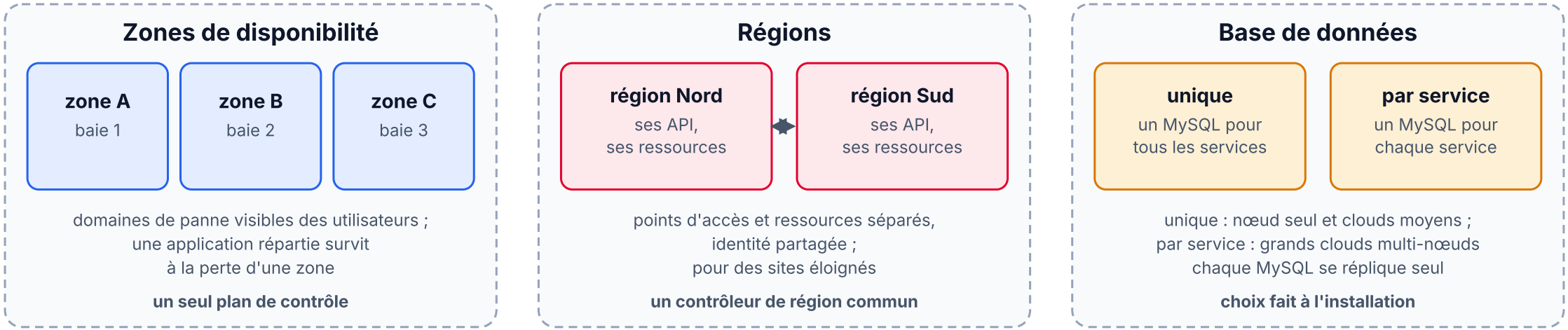
Sécurité et intégrations d'entreprise

Sujet	Ce que propose la distribution	Point d'attention
Chiffrement des points d'accès	**TLS par autorité tierce, par autorité interne ou par Vault**	à décider avant d'ouvrir le cloud aux utilisateurs
Annuaire d'entreprise	**intégration LDAP : un domaine Keystone adossé à l'annuaire**	les comptes de service restent dans le domaine local
Secrets	**Vault en charm, Barbican pour les utilisateurs**	Vault se descelle à la main après chaque redémarrage
Accès à Internet	**déploiement derrière un mandataire, avec liste des adresses à autoriser**	le fonctionnement **totalement hors ligne** est annoncé, pas encore disponible
Traçabilité	**journal d'audit des appels d'API**	à envoyer vers l'outil de collecte de l'entreprise
Durcissement	**profils CIS et DISA-STIG, modules FIPS, par Ubuntu Pro**	exige l'abonnement
Correctifs du noyau	**Livepatch : correctifs appliqués sans redémarrage, par Ubuntu Pro**	ne dispense pas des redémarrages planifiés
Isolation des instances	**épinglage de processeurs, grandes pages, périphériques PCI, DPDK**	à déclarer dans le manifeste, nœud par nœud

Confinement par défaut : les snaps sont strictement confinés et les services du plan de contrôle isolés dans leurs pods ; une faille dans un service n'ouvre pas le système hôte.

Logiciels tiers : ce qui parle aux **API** d'OpenStack s'intègre normalement. Ce qui exige un accès aux mécanismes internes relève d'une prestation de Canonical.

Disponibilité, zones, régions



Ce qui est redondé dans un cloud à trois nœuds de contrôle

- **API :** plusieurs pods par service, derrière Traefik et une adresse de service.
- **MySQL et RabbitMQ :** grappes de trois, réplication intégrée.
- **Bases OVN :** grappe de trois.
- **Ceph :** trois copies de chaque donnée, sur trois nœuds.

Ce qui demande une décision

- **Contrôleur Juju et MAAS :** à rendre redondants séparément.
- **Instances :** elles ne sont pas relancées ailleurs par défaut ; activer la **reprise d'instances** si c'est attendu.
- **Très grande taille :** Canonical recommande **plusieurs petits clouds** gérés de façon centralisée plutôt qu'un seul très grand.

05

Avantages, limites, modèle de support

Ce que la stack fait bien, ce qu'elle ne fait pas encore, combien de temps une version est maintenue et ce que l'on peut acheter autour.

- Avantages de la stack
- Cycle de vie des versions
- Services proposés par Canonical
- Quand choisir cette stack
- Limites à connaître
- Niveaux de support et abonnement
- Comparaison avec les alternatives
- Monter une maquette

Avantages de la stack Canonical

Du serveur nu au cloud, un seul fournisseur

MAAS, Juju, Kubernetes, Ceph, OVN, OpenStack et le système viennent du même éditeur, sont testés ensemble et suivent le même calendrier.

Démarrage très court

Un cloud fonctionnel sur une machine en quelques commandes, puis extension par ajout de nœuds, sans réinstallation.

Le même outil à toutes les tailles

De la maquette sur un poste au cloud de plusieurs baies : mêmes commandes, mêmes charms, mêmes procédures.

Peu de décisions à prendre

Hyperviseur, réseau virtuel, stockage, base, bus : les choix sont faits et intégrés. L'équipe se concentre sur l'usage du cloud.

Gratuit sans contrat, supportable ensuite

Le logiciel est le même avec ou sans abonnement. On peut démarrer seul et souscrire plus tard, sans migration.

Versions à très long terme

Une version LTS tous les deux ans : cinq ans de maintenance, jusqu'à douze avec Ubuntu Pro et Legacy Support. Peu de montées de version imposées.

Avantages techniques

- **Plan de contrôle en pods :** redémarrage, répartition et mises à jour progressives assurés par Kubernetes.
- **Services découplés du système hôte :** rocks et snaps embarquent leurs dépendances.
- **Mises à jour transactionnelles** des snaps, avec retour arrière.
- **Relations Juju :** aucune adresse ni mot de passe recopié à la main entre services.

Avantages d'organisation

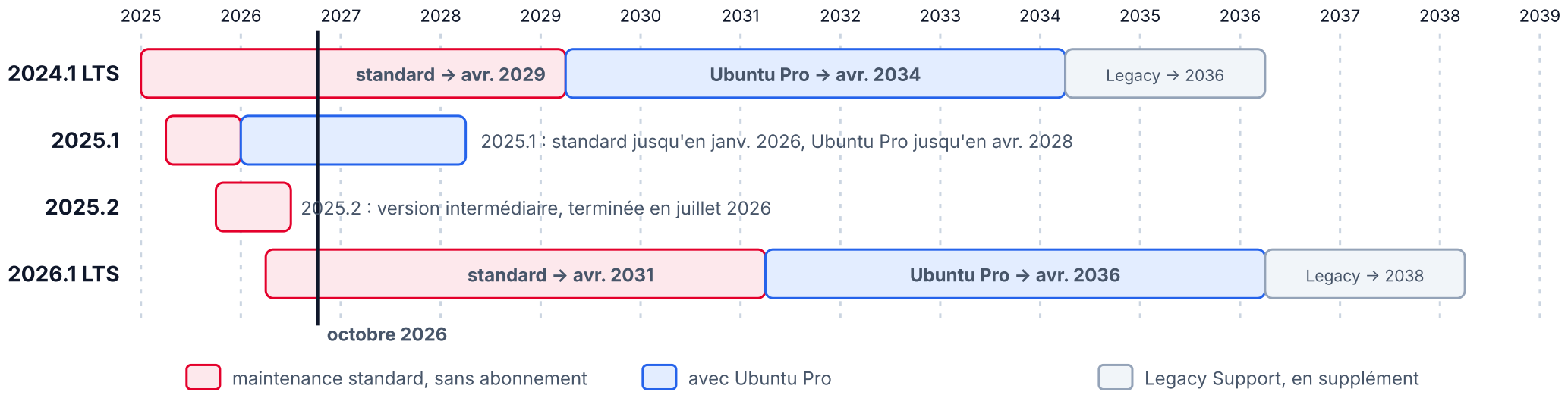
- **Petite équipe possible :** le savoir-faire d'exploitation est dans les charms.
- **Code ouvert et gouverné en amont :** Sunbeam est un projet officiel d'OpenStack.
- **Tarification par nœud** pour l'abonnement, lisible et prévisible.
- **Briques réutilisables** pour d'autres usages : MAAS et Juju servent aussi pour Kubernetes ou des bases de données.

Limites à connaître avant de s'engager

Limite	Détail	Conséquence pratique
Choix techniques imposés	**QEMU/KVM, OVN, Ceph ; en plus, baies Pure Storage, Dell et Hitachi (option)**	une autre baie ou un autre réseau virtuel ne s'intègrent pas aujourd'hui
Écosystème fermé sur Ubuntu	**hôtes Ubuntu, Kubernetes de Canonical, snaps**	inadapté si la politique impose un autre système ou un Kubernetes existant
Architecture amd64 seulement	**pas d'autre architecture pour les nœuds**	à vérifier avant tout achat de matériel
Accès à Internet	**déploiement possible derrière un mandataire ; hors ligne complet pas encore disponible**	frein pour les réseaux strictement isolés
Fournisseur de machines figé	**manuel ou MAAS : choix à la conception, sans retour**	démarrer en manuel puis passer à MAAS impose de redéployer
Montées de version	**la procédure de rafraîchissement ne franchit pas les pistes**	vérifier le chemin de mise à niveau entre versions avant de choisir la version de départ
Peu de réglages exposés	**seules les options prévues par les charms et le manifeste**	les besoins atypiques passent par une prestation ou une contribution
Compétences Juju	**moins répandues qu'Ansible ou Kubernetes**	formation à prévoir ; recrutement plus difficile
Produit jeune	**première version LTS de Sunbeam : 2024.1 ; rôles et procédures évoluent encore**	lire les notes de version et les limites connues à chaque mise à jour

Plusieurs couches à diagnostiquer : une API qui ne répond pas peut venir d'OpenStack, du charm, de Juju, de Kubernetes, de Traefik ou du snap sous-jacent. Savoir descendre les couches une à une est la compétence clé de l'exploitant.

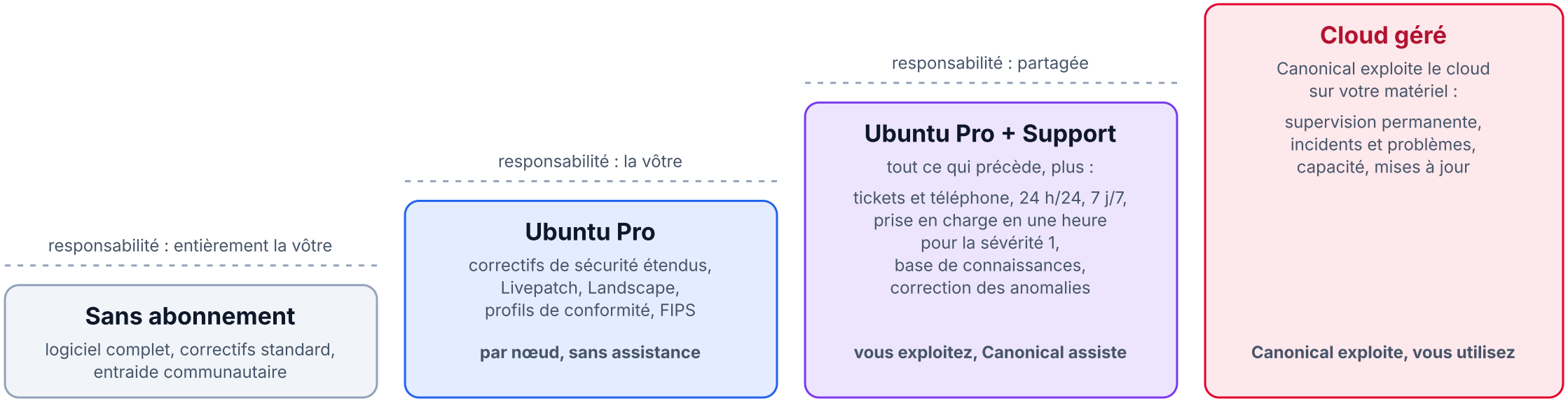
Cycle de vie des versions de Canonical OpenStack



- **Deux versions par an**, en général en avril et en octobre, alignées sur le calendrier d'OpenStack.
- **Versions LTS de Sunbeam :** 2024.1 et 2026.1 ; la prochaine est 2028.1, 2027.1 étant une version intermédiaire.
- **Système hôte :** 2024.1 et 2025.1 sur Ubuntu 24.04 ; 2026.1 sur Ubuntu 24.04 ou 26.04.
- **Maintenance standard :** cinq ans sur une LTS, sans abonnement.
- **Ubuntu Pro :** cinq ans de plus ; **Legacy Support :** deux ans encore, soit douze au total.
- **Versions intermédiaires :** quelques mois seulement. Une production se pose sur une LTS.

> Dates relevées dans la page « Release cycle and supported versions » de la documentation 2026.1 ; la page commerciale de Canonical évoque jusqu'à quinze ans de correctifs de sécurité. Vérifier la disponibilité d'un canal avec `snap info openstack`.

Modèle de support : quatre niveaux d'engagement



- **Deux périmètres d'abonnement :** « Infra-only », qui couvre le système et l'infrastructure, ou complet, qui ajoute les applications. Les deux ouvrent droit au support de Canonical OpenStack.
- **Entre support et cloud géré :** une formule intermédiaire d'astreinte, où les ingénieurs de Canonical interviennent sur vos incidents critiques pendant que votre équipe exploite.
- **Ce que l'abonnement ne remplace pas :** une équipe qui connaît le cloud. Le support traite les anomalies, pas l'exploitation courante.
- **Hors support :** les paquets OpenStack installés à la main depuis les archives reçoivent des correctifs de sécurité, pas d'assistance complète.

> Tarifs et contenu exact des formules : non reproduits ici, ils changent. Demander un devis ; la tarification se fait **par nœud**.

Services proposés par Canonical autour du cloud



L'abonnement Ubuntu Pro est attaché aux nœuds en cours de projet ; chaque étape peut être achetée ou assurée en interne.

Vous avez…	Formule adaptée
une équipe expérimentée, un cloud non critique	sans abonnement, ou Ubuntu Pro seul
une équipe compétente, un cloud de production	Ubuntu Pro + Support
une équipe réduite, un cloud critique	support avec astreinte renforcée
pas d'équipe d'exploitation	cloud géré
un projet à lancer vite	conception et construction forfaitaires, puis reprise en interne

Questions à poser avant de signer

- Quel délai de **prise en charge** et quel délai de **résolution**, par sévérité ?
- Quelles **modifications** font sortir du support : charm épinglé, option non documentée, pilote tiers ?
- Qui fait les **montées de version**, et sont-elles incluses ?
- Quelle **réversibilité** en fin de contrat : le cloud continue-t-il de fonctionner et de recevoir des correctifs ?

> Sur le dernier point, la réponse est favorable : le logiciel reste utilisable sans contrat, avec la maintenance standard.

5 · AVANTAGES, LIMITES, SUPPORT

La stack Canonical face aux alternatives

	Canonical OpenStack (Sunbeam)	Kolla-Ansible	Red Hat RHOSO	Mirantis MOSK
Nature	distribution libre, support optionnel	outil communautaire	produit sous souscription	produit sous souscription
Machines nues	MAAS, facultatif	Kayobe, projet frère	Metal3, dans OpenShift	inclus
Orchestrateur	Juju, derrière `sunbeam`	Ansible	opérateurs OpenShift, Ansible	opérateur, Helm
Socle du plan de contrôle	Kubernetes de Canonical	Docker ou Podman	OpenShift	Kubernetes de Mirantis
Système hôte	Ubuntu	CentOS Stream, Debian, Rocky, Ubuntu	RHEL	Ubuntu
Stockage	Ceph (MicroCeph) ; baies Pure Storage, Dell, Hitachi	Ceph externe, LVM, pilotes tiers	Red Hat Ceph, pilotes certifiés	Ceph (Rook), pilotes tiers
Taille minimale	une machine	une machine	un cluster OpenShift et un nœud de calcul	cluster de gestion et cluster géré
Utilisable sans contrat	oui, intégralement	oui	non	non
Liberté de réglage	limitée aux options des charms	élevée	encadrée	encadrée
Compétence clé	Juju, snaps, un peu de Kubernetes	Ansible, conteneurs	OpenShift	Kubernetes
Durée d'une version	5 ans sur LTS ; 12 avec Ubuntu Pro et Legacy	celle de l'amont, environ 18 mois	plusieurs années, selon contrat	selon contrat

Ce qui distingue Canonical : c'est la seule offre d'éditeur dont le produit complet s'utilise sans contrat, et qui démarre sur une seule machine.

Ce qui la pénalise : le moins de choix techniques et la technologie d'orchestration la moins répandue des quatre.

Quand choisir cette stack — et quand l'éviter

Elle convient bien si…

- Vos serveurs tournent déjà sous **Ubuntu**, ou rien n'impose un autre système.
- Vous partez de **zéro** : pas de réseau virtuel à réutiliser, stockage Ceph ou baie prise en charge.
- L'équipe est **réduite** et veut un résultat rapide avec peu de décisions.
- Vous voulez **démarrer sans budget logiciel** et garder la possibilité de souscrire plus tard.
- Vous cherchez des versions **à très long terme**.
- Le cloud doit pouvoir **commencer petit** : trois nœuds, puis grandir.
- Vous exploitez déjà **MAAS ou Juju** pour autre chose.

Mieux vaut regarder ailleurs si…

- Une des **limites** listées slide 51 est bloquante : baie non prise en charge, autre réseau virtuel, matériel **hors amd64**, réseau **strictement isolé**.
- La politique impose **RHEL**, **OpenShift** ou un Kubernetes déjà en place.
- L'équipe, **experte en Ansible**, veut régler chaque service et embarquer ses propres correctifs.
- Vous cherchez la plus grande **liberté de choix techniques** : Kolla-Ansible ou OpenStack-Ansible s'y prêtent mieux (module 5).

Maquette et formation

Sunbeam sur une machine : le chemin le plus court vers un vrai cloud, fidèle à la production.

Petit cloud de production

Trois nœuds hyperconvergés, fournisseur manuel, abonnement ajouté quand le cloud devient critique.

Cloud de plusieurs baies

MAAS, rôles séparés, réseau à deux niveaux, contrôleurs redondants, manifeste versionné.

Décision à ne pas reporter : manuel ou MAAS, et la version LTS de départ. Ce sont les deux choix sur lesquels on ne revient pas sans redéployer.

Monter une maquette de la stack

Objectif	Maquette	Ressources
Découvrir OpenStack	**Sunbeam, nœud unique, fournisseur manuel**	1 machine : 4 cœurs, 16 Gio, 100 Gio
Comprendre MAAS	**MAAS en machine virtuelle, deux ou trois machines virtuelles à amorcer par le réseau**	8 à 16 Gio au total
Comprendre Juju	**Juju sur LXD local, puis sur MAAS**	8 Gio
Voir la haute disponibilité	**Sunbeam à trois nœuds**	3 machines de 16 cœurs, 32 Gio, 500 Gio
La chaîne complète	**MAAS, puis Sunbeam avec le fournisseur MAAS**	un serveur de laboratoire dédié

Ordre conseillé : Sunbeam à nœud unique d'abord, pour voir le résultat ; puis MAAS et Juju séparément, pour comprendre ce que Sunbeam fait à votre place.

Points d'attention sur un poste de travail

- **Virtualisation imbriquée :** à activer sur l'hyperviseur du poste pour que les instances utilisent KVM ; sinon elles tournent en émulation, très lentement.
- **Réseau d'amorçage pour MAAS :** un réseau interne **sans** le serveur DHCP de l'outil de virtualisation ; les machines virtuelles démarrent sur le réseau en premier.
- **Pilotage de l'alimentation :** sans carte de gestion, déclarer le type « manuel » dans MAAS et démarrer les machines virtuelles à la main.
- **Disques pour Ceph :** ajouter un second disque virtuel, vierge, à chaque nœud de stockage.
- **Accès à Internet :** indispensable pendant l'installation — snaps, charms et images sont téléchargés.
- **Nom de machine :** pleinement qualifié, résolu localement, fixé avant l'installation.

Ne pas extrapoler : une maquette prouve le fonctionnement et les procédures, pas les performances ni la tenue à la panne.

SYNTHÈSE

Ce qu'il faut retenir

MAAS

- Fait d'un parc de serveurs un cloud de machines nues.
- Région pour l'état et l'API, rack pour l'amorçage et l'alimentation.
- New → Commissioning → Ready → Deployed.
- Étiquettes, zones et espaces : le vocabulaire des couches supérieures.

Juju

- Déploie et relie des applications par des charms.
- Contrôleur, modèle, application, unité, relation.
- Même outil sur machines et sur Kubernetes.
- `juju status` : le premier réflexe de diagnostic.

Canonical OpenStack

- Sunbeam : plan de contrôle en pods, snaps sur les hôtes.
- Rôles control, compute, network, storage.
- Fournisseur manuel ou MAAS, choisi à la conception.
- KVM et OVN imposés ; Ceph, plus quelques baies tierces.

Question	Réponse courte
Qu'apporte la stack ?	**une chaîne complète et cohérente, du serveur nu au cloud, utilisable sans contrat et dès une machine**
Que coûte-t-elle en liberté ?	des choix techniques imposés et peu de réglages hors de ce que prévoient les charms
Combien de temps dure une version ?	cinq ans en standard sur une LTS, jusqu'à douze avec Ubuntu Pro et le support étendu
Quel support ?	aucun, abonnement seul, abonnement avec assistance permanente, ou cloud entièrement géré — par nœud
Quelles décisions sont définitives ?	le fournisseur de machines et la version de départ

Sources : documentation de Canonical OpenStack 2026.1 (architecture, conception, prérequis, installation, mises à jour, cycle de vie), documentation de MAAS 3.7, notes de version de Juju 4.0, page de support OpenStack de Canonical — consultées en octobre 2026.